

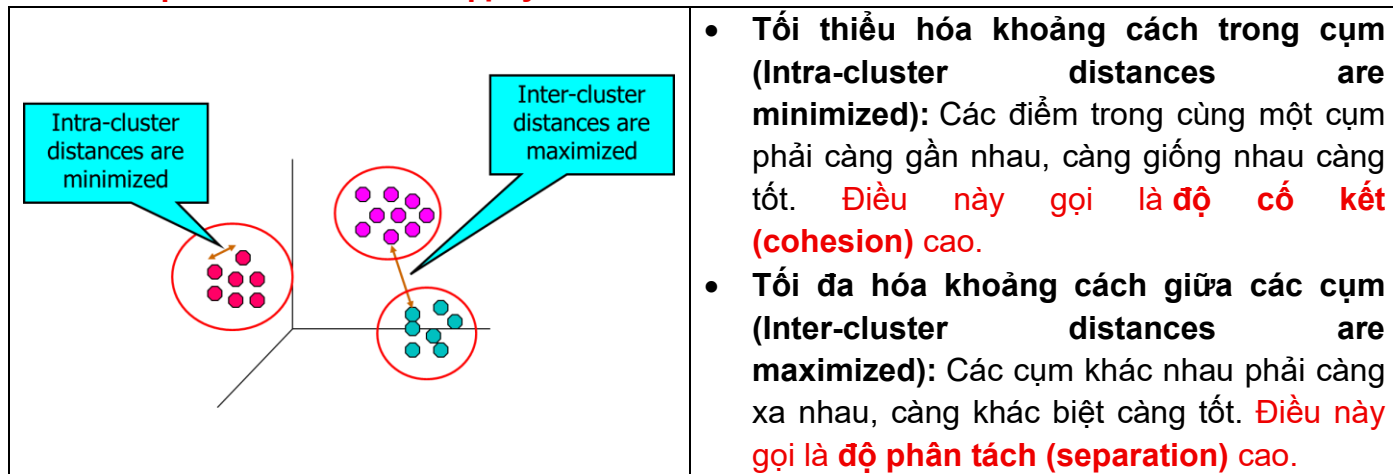
CHAPTER 5: DATA CLUSTERING (PHÂN CỤM DỮ LIỆU)

Mục lục

5.1. Cluster Analysis (Phân tích Cụm).....	2
5.2. Types of Clusterings (Các kiểu Phân cụm).....	6
5.3. Types of Clusters (Các loại Cụm)	8
5.4. Characteristics of the Input Data Are Important.....	9
5.5. Clustering Algorithms (Các Thuật toán Phân cụm)	10
5.5.1. K-means Clustering	10
5.5.1.1. Techniques to find Optimal value of k	11
5.5.1.1.1. Phương pháp Elbow (Khuỷu tay)	12
5.5.1.1.2. Phương pháp Silhouette (Độ tương đồng).....	12
5.5.1.1.3. Phương pháp Gap Statistic	13
5.5.1.1.4. Davies-Bouldin Index	14
5.5.1.2. K-means Clustering – Details.....	15
5.5.1.3. K-means Objective Function (Hàm mục tiêu của K-means)	15
5.5.1.4. Importance of Choosing Initial Centroids (Tầm quan trọng của việc chọn Tâm ban đầu)	16
5.5.1.5. K-means++	17
5.5.1.6. K-medoids: PAM (Partitioning Around Medoids)	17
5.5.1.7. Bisecting K-means	19
5.5.1.8. Limitations of K-means (Hạn chế của K-means).....	19
5.5.2. Hierarchical Clustering	20
5.5.2.1. Agglomerative Clustering Algorithm (Thuật toán Tích tụ)/ AGNES (Agglomerative Nesting)	22
5.5.2.2. Divisive Clustering Algorithm (Thuật toán Phân cụm Phân chia)/ DIANA - DIvisive ANALysis	28
5.5.2.3. Time and Space requirements, Problems and Limitations	28
5.5.3. Density-Based Clustering	29
5.5.3.1. DBSCAN (Density-Based Spatial Clustering of Applications with Noise).....	29
5.5.3.2. DBSCAN Algorithm	31
5.5.3.2. Ưu và Nhược điểm của DBSCAN.....	33
5.5.3.3. Determining EPS and MinPts (Cách chọn tham số)	34
5.6. Cluster Validity (Đánh giá Cụm).....	36
5.6.1. Measures of Cluster Validity (Các độ đo Đánh giá Cụm).....	37
5.6.1.1. Unsupervised Measures: Cohesion and Separation	37
5.6.1.2. Unsupervised Measures: Silhouette Coefficient.....	38
5.6.1.3. External index: Entropy, Purity	39
5.6.2. Measuring Cluster Validity Via Correlation (Đo lường bằng Tương quan)	41
5.6.3. Statistical Framework (Khung đánh giá Thống kê).....	42
5.6.4. Final Comment on Cluster Validity.....	43

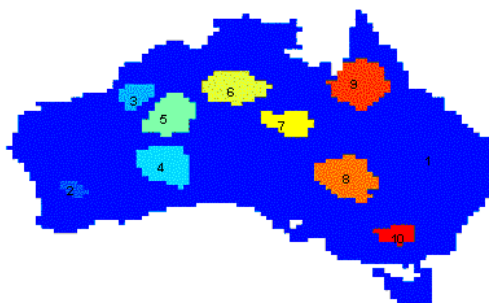
5.1. Cluster Analysis (Phân tích Cụm)

- **Cluster (cụm/ nhóm/ lớp):** Cho một tập các đối tượng, hãy đặt chúng vào các nhóm sao cho các đối tượng trong cùng một nhóm thì tương đồng/liên quan với nhau và khác biệt với các đối tượng trong các nhóm khác.
- **Gom cụm:** Tìm sự tương đồng giữa dữ liệu theo các đặc điểm được tìm thấy trong dữ liệu và nhóm các đối tượng dữ liệu tương đồng thành các cụm.
- **Gom cụm như thế nào là hợp lý?**



- Cho CSDL $D = \{t_1, t_2, \dots, t_n\}$ và số nguyên k .
Gom cụm là bài toán xác định ánh xạ $f: D \rightarrow \{1, \dots, k\}$ sao cho mỗi t_i được gán vào một nhóm K_j với $1 \leq j \leq k$
- **Học không giám sát:** không có lớp được xác định trước (nghĩa là học bằng cách quan sát so với học bằng ví dụ: được giám sát).
- Mục đích cốt lõi của phân cụm (Cluster Analysis) là **tự động khám phá ra các nhóm (cụm) tự nhiên trong dữ liệu mà không cần biết trước các nhãn**. Nói cách khác, nó giúp chúng ta tìm ra "cấu trúc ẩn" hoặc các quy luật ngầm trong một tập dữ liệu thô.
- Mục đích này có thể được chia thành hai mục tiêu chính:
 - **Khám phá và Hiểu biết (Discovery and Understanding):** Đây là mục tiêu quan trọng nhất, nơi chúng ta sử dụng phân cụm như một công cụ thám hiểm để tìm ra những hiểu biết mới từ dữ liệu.
 - **Tìm ra các nhóm có ý nghĩa:** Phân cụm giúp xác định các nhóm đối tượng có chung đặc điểm mà trước đây chúng ta chưa hề biết.
 - **Tạo ra giả thuyết:** Kết quả phân cụm có thể gợi ý những giả thuyết mới để kiểm chứng. Ví dụ, tại sao nhóm khách hàng này lại mua sắm nhiều vào ban đêm?
 - **Phát hiện sự bất thường (Anomaly Detection):** Những đối tượng không thuộc về bất kỳ cụm nào (outliers) thường là những trường hợp đặc biệt cần chú ý, chẳng hạn như giao dịch gian lận, lỗi hệ thống, hoặc một ca bệnh hiếm.
 - **Ví dụ thực tế:**
 - **Phân khúc khách hàng (Customer Segmentation):** Một công ty bán lẻ có thể phân cụm khách hàng dựa trên lịch sử mua sắm, hành vi duyệt web, và thông tin nhân khẩu học. Kết quả có thể là các nhóm như: "Người săn khuyến mãi", "Khách hàng trung thành có thu nhập cao", "Người mua sắm ngẫu nhiên". Từ đó, công ty có thể tạo ra các chiến dịch marketing nhắm mục tiêu hiệu quả hơn.
 - **Phân tích sinh học:** Các nhà khoa học có thể phân cụm các gen dựa trên mức độ biểu hiện của chúng trong các điều kiện khác nhau. Các gen rơi vào cùng một cụm có thể có chức năng sinh học liên quan đến nhau, giúp khám phá ra các con đường sinh hóa mới.

- **Phân tích mạng xã hội:** Phân cụm người dùng trên Facebook hoặc Twitter có thể phát hiện ra các "cộng đồng" có cùng sở thích, quan điểm chính trị, hoặc mối quan tâm, dù họ không trực tiếp kết nối với nhau.
- **Tổ chức và Tóm tắt (Organization and Summarization):** Khi đối mặt với một lượng dữ liệu khổng lồ, phân cụm giúp đơn giản hóa và cấu trúc lại dữ liệu để dễ dàng quản lý và sử dụng hơn.
 - **Giảm độ phức tạp:** Thay vì phải làm việc với hàng triệu điểm dữ liệu riêng lẻ, chúng ta có thể **làm việc với một vài "cụm" đại diện**.
 - **Nén dữ liệu:** Thông tin có thể được tóm tắt bằng cách **chỉ lưu trữ tâm (centroid) của các cụm và nhãn cụm** cho mỗi điểm, giúp giảm kích thước lưu trữ.
 - **Tiền xử lý cho các thuật toán khác:** Kết quả phân cụm có thể được **sử dụng làm đầu vào cho các thuật toán học máy khác** (ví dụ: phân loại hoặc hồi quy). Thay vì sử dụng các thuộc tính gốc, ta có thể sử dụng "ID của cụm" làm một thuộc tính mới.
 - **Ví dụ thực tế:**
 - **Tổ chức tài liệu:** Google News tự động phân cụm hàng ngàn bài báo từ các nguồn khác nhau thành các sự kiện tin tức cụ thể (ví dụ: tất cả các bài báo về một trận bóng đá sẽ được gom vào một nhóm).
 - **Phân vùng hình ảnh (Image Segmentation):** Trong xử lý ảnh, phân cụm được dùng để nhóm các pixel có màu sắc và kết cấu tương tự nhau thành các vùng riêng biệt (ví dụ: bầu trời, cây cối, tòa nhà). Đây là bước quan trọng để nhận dạng vật thể.
 - **Hệ thống gợi ý (Recommender Systems):** Phân cụm người dùng có cùng sở thích xem phim. Nếu bạn và một nhóm người khác cùng thích phim A, B, C, và nhóm đó cũng thích phim D, hệ thống có thể sẽ gợi ý phim D cho bạn.
- **Applications of Cluster Analysis (Ứng dụng của Phân tích Cụm)**
 - **Understanding (Hiểu biết)**
 - Phân cụm giúp **khám phá ra cấu trúc tiềm ẩn** trong dữ liệu. Ví dụ:
 - **Tài liệu:** Gom các bài báo có cùng chủ đề lại với nhau (như Google News).
 - **Sinh học:** Nhóm các gen có cùng chức năng dựa trên biểu hiện của chúng.
 - **Kinh tế:** Nhóm các cổ phiếu có biến động giá tương tự nhau để phân tích danh mục đầu tư.
 - **Summarization**
 - **Reduce the size** of large data sets → Giảm độ phức tạp của tập dữ liệu. Ví dụ:
 - **Xử lý ảnh:** Thay vì lưu trữ hàng triệu pixel, ta có thể phân cụm chúng thành các vùng màu (ví dụ: bầu trời, cây cối, mặt đất) để nén ảnh.



- **Dữ liệu không gian:** Hình ảnh bản đồ Úc phân cụm lượng mưa (precipitation) cho thấy thay vì xem xét dữ liệu từ hàng ngàn trạm thời tiết, ta có thể tóm tắt thành một vài "vùng khí hậu" chính.

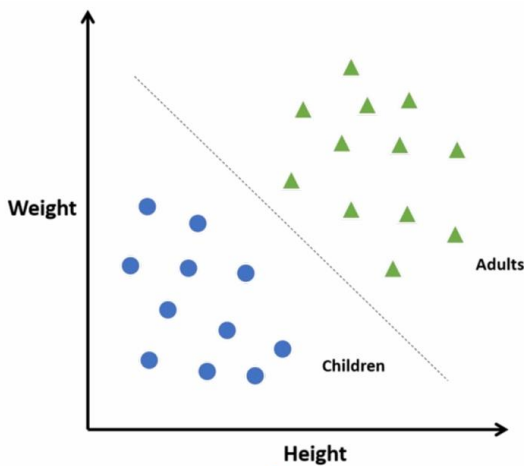
- **Gom cụm vs. Phân lớp**

- **Gom cụm truyền thống**

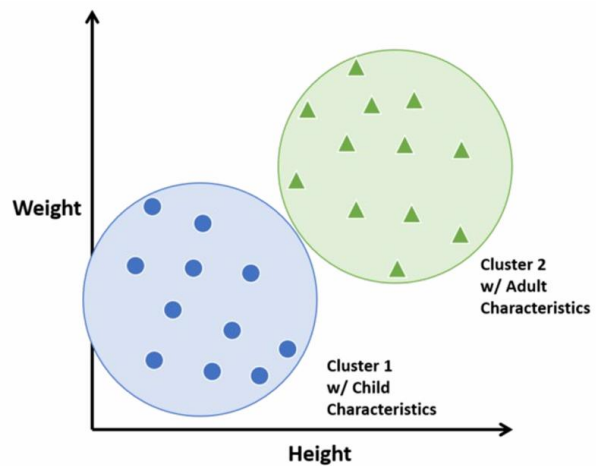
- Mục tiêu: xác định các cụm đối tượng tương tự. **Các cụm được phát hiện**
 - **Bộ dữ liệu gồm các thuộc tính**
 - Không giám sát (nhãn lớp phải học)
 - Đánh giá tính tương đồng, “hàm khoảng cách” là rất quan trọng, bởi vì các cụm được phát hiện dựa trên khoảng cách / mật độ.

- **Phân lớp**

- Mục tiêu là dự đoán các lớp từ các thuộc tính/giá trị thuộc tính của đối tượng. **Các lớp được xác định trước**
 - **Bộ dữ liệu gồm các thuộc tính và một nhãn lớp**
 - Được giám sát (nhãn lớp đã biết)
 - Bộ phân lớp được học từ các tập ví dụ đã phân lớp
 - Bộ phân lớp cần có độ chính xác cao

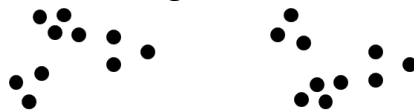


Tìm phương pháp để dự đoán lớp của mẫu mới từ các mẫu đã được gán nhãn lớp trước



Tìm các cụm/ nhóm “tự nhiên” của các mẫu chưa được gán nhãn

- **A Cluster can be Ambiguous**



How many clusters?



Six Clusters



Two Clusters



Four Clusters

- **Không có một câu trả lời "đúng" duy nhất cho câu hỏi "Có bao nhiêu cụm?".** Việc phân cụm phụ thuộc vào:

- **Mức độ chi tiết (granularity)** mà chúng ta muốn xem xét.
- **Thuật toán** được sử dụng.
- **Ngữ cảnh** của bài toán. Ví dụ, các điểm đó có thể là vị trí các cửa hàng. Phân thành 2 cụm có thể là "khu vực phía Bắc" và "khu vực phía Nam". Phân thành 6 cụm có thể là các cụm cửa hàng nhỏ hơn trong từng quận.

- **Phương pháp gom cụm tốt: tạo ra các cụm có chất lượng**

- Giữa các đối tượng trong cùng 1 cụm sự giống nhau cao (*intra class*)
- Giữa các cụm sự giống nhau thấp (*inter-class*)
- Chất lượng của kết quả gom cụm phụ thuộc vào:
 - Độ đo sự tương tự
 - Thuật toán gom nhóm
 - Khả năng phát hiện một vài hay tất cả các mẫu bị che (hidden patterns)

- **Độ đo khoảng cách:** Thường được dùng để xác định sự khác nhau hay giống nhau giữa 2 đối tượng.

- **Khoảng cách Minkowski**

$$d(i, j) = \sqrt[q]{(|x_{i1} - x_{j1}|^q + |x_{i2} - x_{j2}|^q + \dots + |x_{ip} - x_{jp}|^q)} \quad (1)$$

- $i = (x_{i1}, x_{i2}, \dots, x_{ip})$ và $j = (x_{j1}, x_{j2}, \dots, x_{jp})$: 2 đối tượng p-chiều
- q: số nguyên dương

- **Khoảng cách Manhattan (q=1)**

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ip} - x_{jp}| \quad (2)$$

- **Khoảng cách Euclide (q=2)**

$$d(i, j) = \sqrt{(|x_{i1} - x_{j1}|^2 + |x_{i2} - x_{j2}|^2 + \dots + |x_{ip} - x_{jp}|^2)} \quad (3)$$

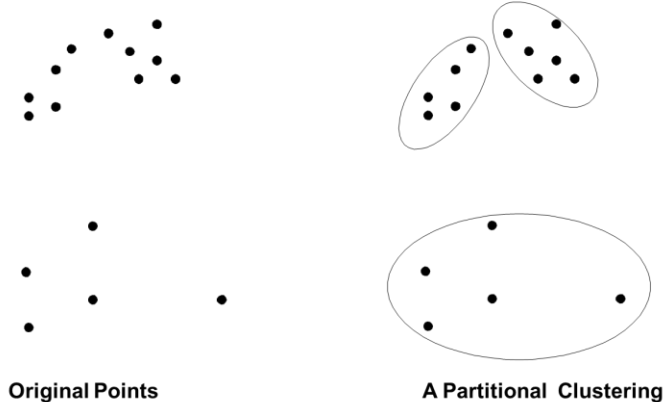
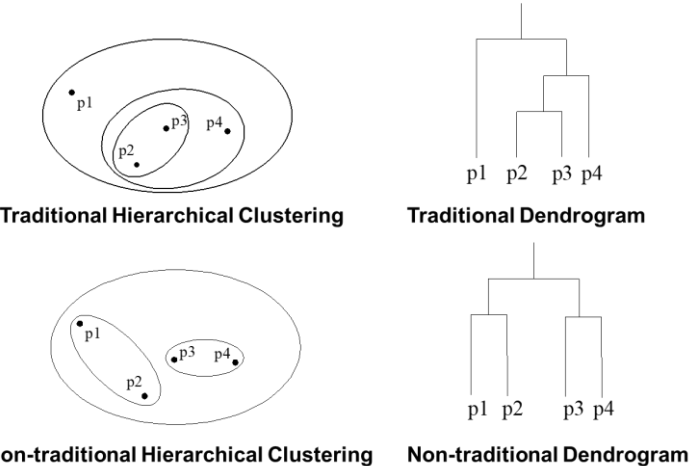
- **Tính chất của độ đo khoảng cách:**

$$d(i, j) \geq 0 \quad d(i, i) = 0 \quad d(i, j) = d(j, i) \quad d(i, j) \leq d(i, k) + d(k, j)$$

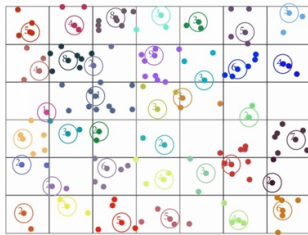
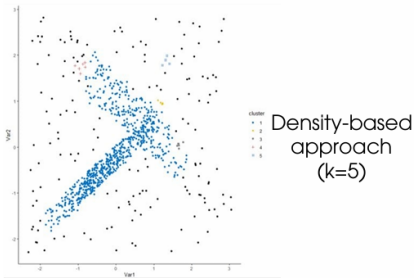
- **Các kiểu dữ liệu khác nhau: yêu cầu độ đo tương đồng cũng sẽ khác nhau**
 - Các biến tỷ lệ theo khoảng: Khoảng cách Euclide
 - Các biến nhị phân: Hệ số so khớp, hệ số Jaccard
 - Các biến tên, thứ tự, tỷ lệ: Khoảng cách Minkowski
 - Các biến dạng hỗn hợp: Công thức trọng lượng

5.2. Types of Clusterings (Các kiểu Phân cụm)

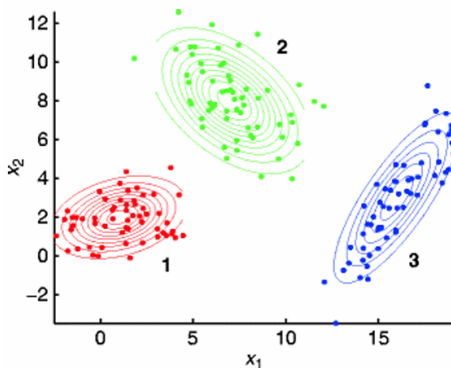
- A **clustering** is a set of **clusters**.
- Hai loại phân cụm chính:

Partitional Clustering (Phân hoạch): + A division data objects into subsets (clusters) such that each data object is in exactly one subset. + Chia dữ liệu thành các tập con không giao nhau (non-overlapping). + Mỗi đối tượng chỉ thuộc về một cụm duy nhất. Không có điểm nào thuộc cả hai nhóm. + Ví dụ: k-means, k-medoids, CLARANS.	 Original Points A Partitional Clustering
Hierarchical Clustering (Phân cấp): + Tạo ra một tập hợp các cụm lồng nhau được tổ chức như một cây phả hệ (dendrogram). + Nó cho thấy mối quan hệ cha-con giữa các cụm. + Dendrogram: Là cách biểu diễn dạng cây của cấu trúc lồng nhau. ○ Các nhánh được hợp nhất ở mức độ nào thì khoảng cách giữa chúng là bấy nhiêu. ○ Ta có thể "cắt" cây ở một độ cao bất kỳ để có được một số lượng cụm mong muốn. + VD: Diana, Agnes, BIRCH, CAMELEON	 Traditional Hierarchical Clustering Traditional Dendrogram Non-traditional Hierarchical Clustering Non-traditional Dendrogram

- Ngoài ra còn có những kiểu sau:



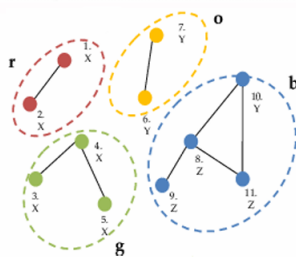
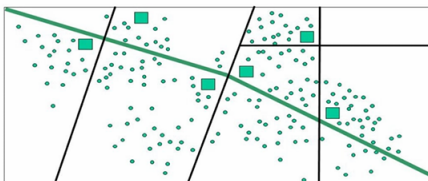
Grid-based approach (k=37)



Model-based

Constraint-Based Clustering Analysis

Clustering analysis: Less parameters but more user-desired constraints, e.g. an ATM allocation problem.



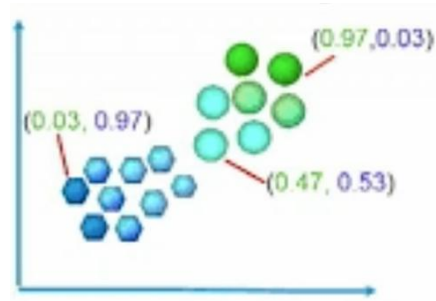
Link-based

- **Phương pháp dựa trên mật độ (Density-based approach):**
 - Dựa trên các chức năng kết nối và mật độ
 - VD: DBSCAN, OPTICS, DenClue
- **Phương pháp dựa trên lưới (Grid-based approach):**
 - Dựa trên cấu trúc chi tiết multiple-level
 - VD: STING, WaveCluster, CLIQUE
- **Phương pháp dựa trên mô hình (Model-based):**
 - Một mô hình được đưa ra giả thuyết cho từng cụm và cố gắng tìm ra sự phù hợp nhất của mô hình đó
 - VD: EM, SOM, COBWEB
- **Phương pháp dựa trên tập phổ biến (Frequent pattern-based):**
 - Dựa trên phân tích các mẫu phổ biến
 - VD: p-Cluster
- **Phương pháp dựa trên ràng buộc (User-guided or constraint-based):**
 - Bằng cách xem xét các ràng buộc do người dùng chỉ định
 - VD: COD (obstacles), constrained clustering
- **Phân cụm dựa trên liên kết (link-based):**
 - Các đối tượng thường được liên kết với nhau theo nhiều cách khác nhau
 - VD: SimRank, LinkClus

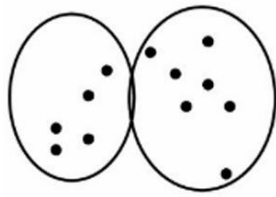
Other Distinctions Between Sets of Clusters

○ Exclusive (Loại trừ) vs. Non-exclusive (Không loại trừ):

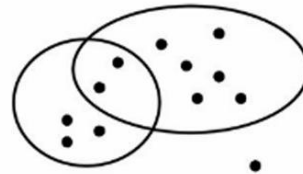
- **Exclusive (or non-overlapping):** Mỗi điểm chỉ thuộc một cụm (như K-means).
- **Non-exclusive (or overlapping):** Một điểm có thể thuộc nhiều cụm.
 - Can belong to multiple classes or could be 'border' points
 - Ví dụ nổi bật là **Fuzzy Clustering**, trong đó mỗi điểm có một "trọng số" hay "xác suất" thuộc về mỗi cụm (between 0 and 1 → Weights must sum to 1). Điều này hữu ích cho các điểm nằm ở biên giới giữa các cụm.



- **Partial (Bộ phận) vs. Complete (Toàn bộ):**
 - **Complete:** Mọi điểm đều được gán vào một cụm nào đó.
 - **Partial:** Chỉ một phần dữ liệu được phân cụm, cho phép bỏ qua các điểm nhiễu (outliers). Thuật toán DBSCAN là ví dụ điển hình cho phương pháp này.



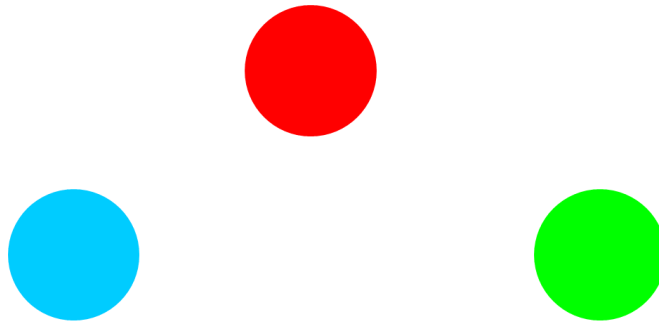
Complete clustering



Partial clustering

5.3. Types of Clusters (Các loại Cụm)

- **Well-separated Clusters:** Cụm được phân tách tốt. Đây là trường hợp lý tưởng nhưng hiếm gặp, nơi mỗi điểm trong cụm gần với mọi điểm khác trong cụm hơn là bất kỳ điểm nào bên ngoài cụm.



3 well-separated clusters

- **Prototype-based (Center-based):** Cụm dựa trên mẫu. Mỗi cụm có một "trung tâm" đại diện. Các điểm được gán vào cụm có trung tâm gần nhất (*closer (more similar) to the prototype or "center" of a cluster, than to the center of any other cluster*). Trung tâm có thể là:
 - **Centroid:** Trung bình của tất cả các điểm trong cụm (không nhất thiết là một điểm dữ liệu thực tế, nghĩa là điểm trung tâm này không buộc phải là một điểm của cụm).
 - **Medoid:** Một điểm dữ liệu thực tế trong cụm mà có khoảng cách trung bình đến các điểm khác là nhỏ nhất (the most "representative" point of a cluster).
- **Contiguity-based (Nearest neighbor or Transitive) (Cụm lân cận):** Cụm dựa trên sự liên tục.
 - Một điểm thuộc về một cụm nếu nó gần ít nhất một điểm khác trong cụm đó.
 - Sự "gần" này thường được đo bằng khoảng cách (ví dụ: khoảng cách Euclidean).
 - Loại này có thể phát hiện các cụm có hình dạng kéo dài, phức tạp.

- **Density-based:** Cụm dựa trên mật độ.
 - Cụm là một vùng có **mật độ điểm dữ liệu cao**, được ngăn cách với các cụm khác bởi các vùng có mật độ thấp.
 - Rất hiệu quả với các cụm hình dạng tùy ý và có khả năng loại bỏ nhiễu.
 - Phương pháp này có thể tìm ra các cụm có hình dạng bất kỳ và có thể loại bỏ các điểm nhiễu (outlier) vì chúng thường nằm trong vùng có mật độ thấp.
- **Objective Function:** Cụm được **định nghĩa bởi một hàm mục tiêu**.
 - Phân cụm trở thành một bài toán tối ưu: **tìm cách phân chia các điểm để tối ưu** (tối thiểu hoặc tối đa) **một hàm số nào đó** (ví dụ: tối thiểu hóa tổng bình phương sai số SSE trong K-means).
 - Slide cũng đề cập đây là bài toán NP-Hard, nghĩa là không thể tìm lời giải tối ưu toàn cục một cách hiệu quả cho dữ liệu lớn.
 - **Can have global or local objectives.**
 - Hierarchical clustering algorithms typically have **local objectives**
 - Partitional algorithms typically have **global objectives**

5.4. Characteristics of the Input Data Are Important

- Khi ta phân cụm, mục tiêu là **gom các điểm giống nhau vào cùng một nhóm và tách các điểm khác biệt ra nhóm khác**.
- Để làm được điều đó, ta phải đo xem **hai điểm dữ liệu giống nhau đến mức nào** → gọi là **proximity measure** (thước đo độ gần/tương đồng).
- Hoặc ta dựa vào **độ dày đặc của dữ liệu** trong một vùng → gọi là **density measure** (mật độ).
→ **Cách đo lường này rất quan trọng** vì nó sẽ quyết định cách dữ liệu được gom thành cụm.

- **Nhưng việc đo “độ gần” hay “mật độ” phụ thuộc vào dữ liệu. Những yếu tố ảnh hưởng:**
 - **Dimensionality (Số chiều dữ liệu):** Nếu dữ liệu **có quá nhiều đặc trưng (feature)**, không gian trở nên "rộng" → **khoảng cách giữa các điểm trở nên ít ý nghĩa** → cần normalization hoặc giảm chiều (PCA).
 - **Sparseness (Độ thưa dữ liệu):** Dữ liệu thưa (nhiều giá trị 0) như dữ liệu văn bản → **không thể áp dụng khoảng cách Euclidean thông thường** → phải dùng cosine similarity.
 - **Attribute Type (Loại thuộc tính):** Dữ liệu số khác dữ liệu phân loại (Nam/Nữ) → mỗi loại cần cách đo tương đồng khác nhau.
 - **Special Relationships (Mối quan hệ đặc biệt):** Ví dụ **autocorrelation: điểm dữ liệu có liên quan đến vị trí/lần cận thời gian trước đó** → như dữ liệu thời tiết, dữ liệu tín hiệu. → phải dùng thuật toán hỗ trợ chuỗi thời gian.
 - **Distribution (Phân bố dữ liệu):** Nếu dữ liệu phân bố không đều, các cụm sẽ có hình dạng và kích thước khác nhau → thuật toán như **DBSCAN** phù hợp hơn **k-means** (vì k-means thích các cụm tròn và đều).

- **Noise and Outliers (Nhiều & điểm bất thường)**
 - Dữ liệu thật luôn có **giá trị lệch** hoặc **nhiều**.
 - Những điểm này **làm sai lệch kết quả phân cụm**, ví dụ:
 - Có thể làm sai lệch vị trí của centroid trong K-means, có thể kéo tâm cụm lệch đi.
 - DBSCAN lại xử lý tốt vì nó đánh dấu chúng là *noise*.

- Clusters khác nhau có thể có:

Đặc điểm	Ý nghĩa
Kích thước khác nhau	Có cụm nhiều điểm, có cụm ít điểm
Mật độ khác nhau	Có cụm điểm dày, có cụm điểm thưa
Hình dạng khác nhau	Không phải lúc nào cũng là hình tròn hoặc elip

→ Không phải thuật toán nào cũng xử lý tốt mọi trường hợp. Các thuật toán khác nhau có thể mạnh với các loại cụm khác nhau. K-means giả định các cụm hình cầu, kích thước và mật độ tương đương. DBSCAN thì không.

- Tóm lại:

- Để phân cụm hiệu quả, ta phải **chọn cách đo khoảng cách/mật độ phù hợp với loại dữ liệu**.
- Dữ liệu có thể **nhiều chiều, có nhiều, phân bố không đều**, khiến việc phân cụm trở nên thách thức.
- **Không có thuật toán clustering nào phù hợp cho mọi loại dữ liệu.**

5.5. Clustering Algorithms (Các Thuật toán Phân cụm)

5.5.1. K-means Clustering

- Đây là thuật toán **phân hoạch**.
- Phải **chỉ định trước số cụm (K)**. Phân hoạch CSDL gồm n đối tượng thành k ($k < n$) cụm.
- Mỗi cụm có một **centroid (tâm)** đại diện. Centroid không nhất thiết phải là một điểm dữ liệu có sẵn.
- Each point is assigned to the cluster with the **closest centroid**
- The objective is find K centroids and the assignment of points to clusters/centroids so as to minimize the sum of distances of the points to their respective centroid.
- **K-means Algorithm** also known as **Lloyd's algorithm**. K-means is sometimes synonymous with this algorithm
- **Thuật toán cơ bản:**

1. Chọn K điểm làm tâm ban đầu của k cụm.
2. **Lặp lại:**
 - Gán mỗi điểm dữ liệu vào cụm có tâm gần nhất (dựa trên khoảng cách Euclide)
 - Tính toán lại giá trị tâm của mỗi cụm (bằng cách lấy trung bình các điểm trong cụm).
3. **Cho đến khi** các tâm không còn thay đổi.

- **Ví dụ:** Một tập dữ liệu nhỏ trong không gian 2D: $X: \{(1,1), (2,1), (4,3), (5,3), (5,4)\}$, nên $n = 5$ (số phần tử), $d = 2$ (số chiều).
 - Chọn $k = 2$, số lần lặp tối đa = 10.
 - **Bước 1: Khởi tạo:** Chọn ngẫu nhiên 2 điểm làm tâm cụm ban đầu, ví dụ: $C_1 = (1,1)$, $C_2 = (4,3)$.

○ **Bước 2: Lập lần thứ nhất**

▪ **Bước gán nhãn (Assignment Step):**

- Sử dụng khoảng cách Euclidean: $d(x, c) = \sqrt{(x_1 - c_1)^2 + (x_2 - c_2)^2}$
- Tiến hành tính khoảng cách từ từng điểm dữ liệu đến từng điểm tâm cụm. Khoảng cách từ điểm dữ liệu đến tâm cụm nào nhỏ hơn (gần hơn) thì thuộc về cụm đó.
- Với (1,1): d đến $C_1 = 0$, d đến $C_2 = \sqrt{(4-1)^2 + (3-1)^2} = \sqrt{13} \approx 3.6$ → Gán vào cụm 1.
- Với (2,1): d đến $C_1 = 1$, d đến $C_2 \approx 3.2$. → Gán vào cụm 1.
- Với (4,3): d đến $C_1 \approx 3.6$, d đến $C_2 = 0$. → Gán vào cụm 2.
- Với (5,3): d đến $C_1 \approx 4.5$, d đến $C_2 = 1$. → Gán vào cụm 2.
- Với (5,4): d đến $C_1 \approx 5$, d đến $C_2 \approx 1.4$. → Gán vào cụm 2.
- Cụm: Cụm 1 = {(1,1), (2,1)}, Cụm 2 = {(4,3), (5,3), (5,4)}.

▪ **Bước cập nhật tâm (Update Step):** Tâm mới là trung bình cộng của các điểm trong cụm.

- C_1 mới: $\frac{(1,1)+(2,1)}{2} = (1.5,1)$.
- C_2 mới: $\frac{(4,3)+(5,3)+(5,4)}{3} \approx (4.67,3.33)$.
- Vì tọa độ của 2 điểm tâm cụm còn thay đổi, nên tiếp tục thực hiện vòng lặp.

○ **Bước 3: Lập lần thứ hai**

▪ **Bước gán nhãn (sử dụng tâm mới):**

- Với (1,1): d đến $C_1 = 0.5$, d đến $C_2 \approx 4.2$. → Cụm 1.
- Với (2,1): d đến $C_1 = 0.5$, d đến $C_2 \approx 3.8$. → Cụm 1.
- Với (4,3): d đến $C_1 \approx 3.2$, d đến $C_2 \approx 0.7$. → Cụm 2.
- Với (5,3): d đến $C_1 \approx 4$, d đến $C_2 \approx 0.5$. → Cụm 2.
- Với (5,4): d đến $C_1 \approx 4.5$, d đến $C_2 \approx 0.7$. → Cụm 2.
- Cụm: Giống như trước (không thay đổi).

▪ **Bước cập nhật tâm:**

- C_1 : Vẫn (1.5,1).
- C_2 : Vẫn (4.67,3.33).

○ **Bước 4: Hội tụ:** Tâm không thay đổi → Thuật toán hội tụ.

○ **Kết quả:**

- Gán cụm: (1,1) → 1, (2,1) → 1, (4,3) → 2, (5,3) → 2, (5,4) → 2.
- Tâm cuối cùng: $C_1 = (1.5,1)$, $C_2 = (4.67,3.33)$.

5.5.1.1. Techniques to find Optimal value of k

- Nếu chọn k quá nhỏ hoặc quá lớn, bạn sẽ gặp các vấn đề:

k quá nhỏ	Nhiều đối tượng khác biệt bị gộp chung → mất chi tiết. Các cụm có độ phân tán lớn, tâm cụm không đại diện tốt.
k quá lớn	Một cụm bị tách nhỏ không cần thiết → mô hình trở nên “quá chi tiết”, không mang ý nghĩa thực tế.
k đúng	Các điểm trong cụm gần nhau, các cụm tách biệt rõ ràng, mô hình phản ánh đúng cấu trúc dữ liệu.

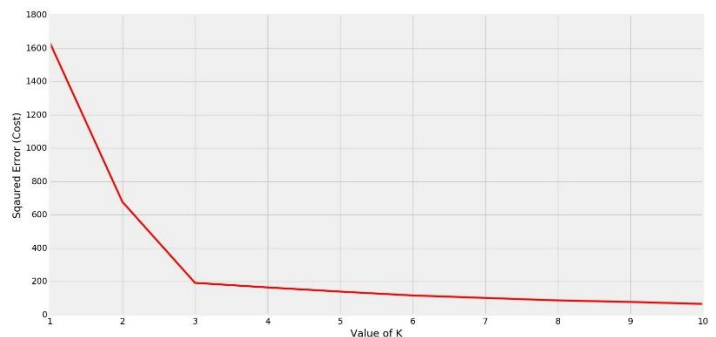
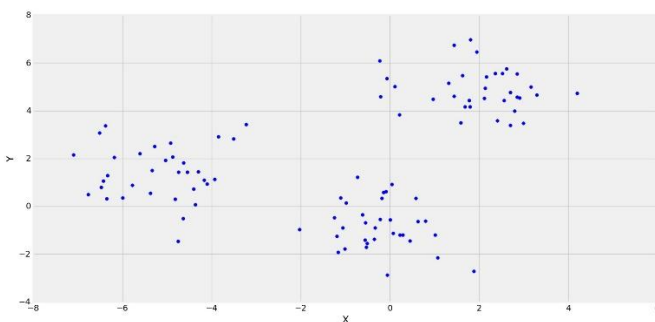
- Nếu dữ liệu Iris thực tế có **3 loại hoa**, mà bạn chọn:
 - $k = 2 \rightarrow$ một loài bị trộn lẫn với loài khác.
 - $k = 5 \rightarrow$ chia một loài thành nhiều nhóm vô nghĩa.
- Choosing the right number of clusters (**K**) in K-Means clustering is very important. If we choose the wrong value of K, the model may not find good patterns in the data. But selecting the best K manually is not easy.
- So we use different techniques to find the optimal value of K. In this article we will learn these techniques.

5.5.1.1.1. Phương pháp Elbow (Khủy tay)

- **Elbow method** is a technique used to choose the best number of clusters for K-Means clustering. The basic idea is to see how the **inertia (cost)** changes as we increase the number of clusters.
 - **Inertia** is the sum of squared distances from each data point to the center of its assigned cluster. It tells us how well the points are grouped in their clusters. The smaller the inertia the better the clustering.
 - Chạy K-Means với các giá trị k khác nhau (ví dụ 1 đến 10).
 - Tính tổng sai số trong cụm (**Within-Cluster Sum of Squares – WCSS**) cho từng k:

$$WCSS = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2, \text{ với } \mu_i \text{ là tâm cụm } i.$$

- We plot the inertia for different values of K, i.e different numbers of clusters. As K increases the inertia decreases because more clusters allow data points to be closer to their cluster centers.
- The goal is to find the value of K where the decrease in inertia starts to slow down. This is like an "elbow" in the plot where the curve bends (cong) and flattens out (phẳng). The number of clusters at this point is considered the optimal value for K.



5.5.1.1.2. Phương pháp Silhouette (Độ tương đồng)

- **Silhouette Method** measures how similar each data point is to its own cluster compared to other clusters (Đo mức độ chặt chẽ và tách biệt giữa các cụm).
- It calculates a score for each point which tells us if the point is well-clustered. A higher score means the point is close to its own cluster and far from others. The Silhouette Score ranges from -1 to +1:

- **+1** means the point is well-placed in its cluster.
- **0** means the point is on the boundary between clusters.
- **-1** means the point is in the wrong cluster.
- We calculate the average Silhouette Score for different values of K. **The best K is the one with the highest average score** indicating that the clusters are well-separated and each point is correctly grouped.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

- $s(i)$ is the silhouette score for point i .
- $a(i)$ is the **average distance from point i to all other points in the same cluster** (cohesion).
- $b(i)$ is the **average distance from point i to all points in the nearest different cluster** (separation).
- **Cons:**
 - Can be **computationally expensive for large datasets.**
 - May not work well if clusters are not spherical or have different densities (Có thể không hoạt động tốt nếu các cụm không phải là hình cầu hoặc có mật độ khác nhau).

5.5.1.1.3. Phương pháp Gap Statistic

- **Gap Statistic** compares the clustering result with a randomly distributed data to determine how well the clustering structure fits the data (So sánh **tổng độ phân tán trong cụm** (WCSS) của dữ liệu thật với dữ liệu **ngẫu nhiên** có cùng kích thước).
 - It measures **the difference** between the total within-cluster variation (**inertia**) of actual data and of random data.
 - The idea is that **the actual clustering should have much lower inertia** compared to random data.
 - **We compute the Gap Statistic for different K values. The optimal K is the one that maximizes the gap, meaning that the clustering is significantly better than random clustering.**
- **Cách làm:**
 - Tạo B bộ dữ liệu ngẫu nhiên (uniform distribution).
 - Tính WCSS cho cả dữ liệu thật và ngẫu nhiên.
 - Tính **Gap**:

$$Gap(K) = \frac{1}{B} \sum_{b=1}^B \log(W_k^b) - \log(W_k)$$

- K is the number of clusters.
- W_k is the within-cluster sum of squares for the **actual** data with K clusters.
- W_k^b is the within-cluster sum of squares for the **randomly** generated data with K clusters.
- B is the number of random data sets used for comparison.
- **Cons:**
 - **Computationally expensive** as it requires generating random datasets and running multiple K-Means clustering operations.
 - May **not perform well on very small datasets.**

5.5.1.1.4. Davies-Bouldin Index

- **Davies-Bouldin Index** evaluates how well-separated the clusters are by *looking at the average similarity between each cluster and its most similar cluster*. (Đánh giá mức độ tương tự giữa các cụm — càng nhỏ càng tốt.)
 - For each pair of clusters it computes **the ratio (tỷ lệ) of sum of within-cluster spread (how tight cluster is) to the distance between the clusters**.
 - A **lower DBI value** means better clustering because it indicates that the clusters are compact and well-separated.
 - We compute the DBI for different values of K and **the best K is the one that gives the lowest DBI**.

$$DBI = \frac{1}{n} \sum_{i=1}^n \max_{j \neq i} \left(\frac{S_i + S_j}{d(c_i, c_j)} \right)$$

- n is the number of clusters.
 - S_i is **the average distance between all points in cluster i and the centroid of cluster i** .
 - $d(c_i, c_j)$ is the distance between the centroids of clusters i and j .
 - $\max_{j \neq i}$ denotes the **maximum value over all other clusters j (excluding i)**.
- **Cons:** It may not work well if the clusters are **highly overlapping (chồng chéo)** or if there's **no clear structure** in the data.

Ngoài ra còn có những phương pháp khác, chẳng hạn:

- **Phương pháp Calinski–Harabasz Index (Variance Ratio Criterion)**
 - Ý tưởng: Tỷ lệ giữa phân tán giữa các cụm và phân tán trong cụm.
 - Công thức:
$$CH(k) = \frac{Tr(B_k)/(k-1)}{Tr(W_k)/(n-k)}$$
 - B_k : ma trận phương sai giữa các cụm
 - W_k : ma trận phương sai trong các cụm
 - n : số điểm dữ liệu
 - Cách chọn: Chạy K-Means với nhiều k , chọn giá trị k cho $CH(k)$ lớn nhất.
- **Phương pháp Information Criterion (BIC hoặc AIC)**
 - Ý tưởng: Xem K-Means như một mô hình xác suất (Gaussian Mixture Model) và chọn k tối ưu theo **BIC** hoặc **AIC** — cân bằng giữa độ khớp và độ phức tạp mô hình.
 - Cách chọn: Chọn k làm cực tiểu AIC/BIC:
$$BIC = -2 \cdot \log\text{-likelihood} + p \cdot \log(n)$$
- **Dựa vào kiến thức chuyên môn (Domain Knowledge)**
 - Ý tưởng: Dựa vào hiểu biết thực tế của bài toán.
 - Ví dụ: nếu bạn phân cụm khách hàng theo hành vi, có thể bạn **biết trước có 3 loại chính**: khách mới, khách trung thành, khách tiềm năng → chọn $k = 3$.

5.5.1.2. K-means Clustering – Details

- **Khởi tạo ngẫu nhiên:** Kết quả phân cụm có thể khác nhau giữa các lần chạy do việc chọn tâm ban đầu là ngẫu nhiên.
- **Hội tụ:** K-means luôn đảm bảo sẽ hội tụ, nhưng nó chỉ hội tụ về một **cực tiểu địa phương (local minimum)**, không chắc là giải pháp tốt nhất toàn cục.
- Hầu hết sự hội tụ xảy ra trong một vài lần lặp đầu tiên.
- **Độ phức tạp (Complexity):** $O(n * K * I * d)$, trong đó n là số điểm, K là số cụm, I là số vòng lặp, và d là số chiều ($k, t \ll n$). Đây là một thuật toán hiệu quả về mặt tính toán (tuyến tính theo n).

5.5.1.3. K-means Objective Function (Hàm mục tiêu của K-means)

- K-means cố gắng tối thiểu hóa hàm **Sum of Squared Error (SSE)** - Tổng bình phương sai số.
 - Given a set X of n objects and an integer K , group the points into K clusters $C = \{C_1, C_2, \dots, C_k\}$:

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} dist^2(m_i, x)$$

- x : một điểm dữ liệu trong cụm C_i
 - m_i : trọng tâm (centroid) hoặc trung điểm (medoid) của cụm C_i
 - k : số cụm
 - $dist()$: khoảng cách Euclidean
- SSE được tính bằng **tổng bình phương khoảng cách Euclidean từ mỗi điểm đến tâm của cụm** mà nó thuộc về.

VD1: Tính SSE của 2 cụm K_1, K_2 với medoid tương ứng $m_1 = 3, m_2 = 4$

Biết: $K_1 = \{2, 3\}$

$K_2 = \{4, 10, 12, 20, 30, 11, 25\}$

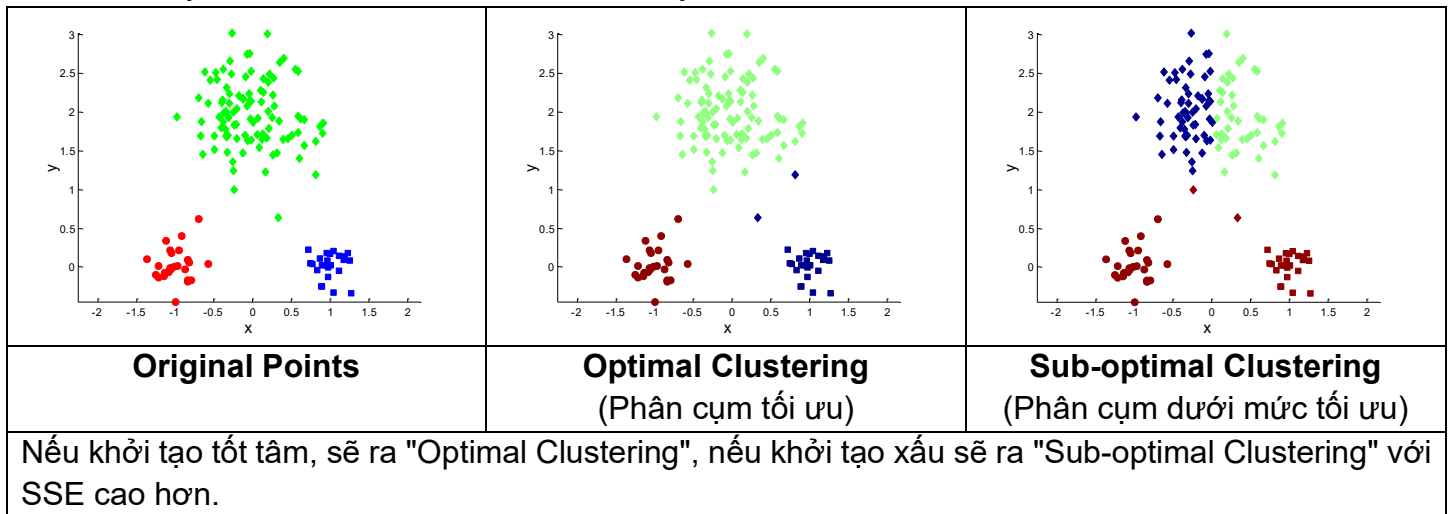
$$SSE = \sum_{i=1}^k \sum_{x \in C_i} dist^2(m_i, x)$$

$$= (3 - 2)^2 + (3 - 3)^2 + (4 - 4)^2 + (4 - 10)^2 + (4 - 12)^2 + (4 - 20)^2 + (4 - 30)^2 + (4 - 11)^2 + (4 - 25)^2 = 1523$$

- Việc gán lại điểm và cập nhật lại tâm ở mỗi bước đều nhằm mục đích làm giảm giá trị SSE. *Thuật toán dừng lại khi SSE không thể giảm thêm được nữa* (đạt cực tiểu địa phương).
- Chính vì hàm mục tiêu này mà K-means có xu hướng tạo ra các cụm đặc, có dạng hình cầu (globular).

5.5.1.4. Importance of Choosing Initial Centroids (Tầm quan trọng của việc chọn Tâm ban đầu)

- Điểm yếu lớn nhất của K-means: sự nhạy cảm với việc khởi tạo tâm.



- **Xác suất chọn được tâm ban đầu tốt trong K-means**
 - **Xác suất P** rằng việc **khởi tạo ngẫu nhiên** K tâm cụm (centroids) trong thuật toán K-Means sẽ **chọn đúng một tâm từ mỗi cụm thực sự** trong dữ liệu.
 - Giả sử:
 - Dữ liệu có K **cụm thực sự** (ground truth clusters).
 - Mỗi cụm có n **điểm** (giả sử các cụm có kích thước bằng nhau để đơn giản).
 - Tổng số điểm: $N = K \cdot n$.
 - Chúng ta **chọn ngẫu nhiên** K điểm từ N điểm làm tâm ban đầu (không lặp).
 - **Câu hỏi: Xác suất để mỗi tâm được chọn thuộc một cụm khác nhau (tức là chọn đúng 1 đại diện từ mỗi cụm)?**

$$P = \frac{\text{number of ways to select one centroid from each cluster}}{\text{number of ways to select } K \text{ centroids}} = \frac{K!n^K}{(Kn)^K} = \frac{K!}{K^K}$$

- P là xác suất rằng tất cả K tâm ban đầu đều nằm ở K cụm khác nhau.
- **Tử số (Numerator):** Số cách chọn **đúng 1 tâm từ mỗi cụm** và **mỗi cụm khác nhau**:
 - $K! \cdot n^K$
 - n^K : Với mỗi cụm, có n điểm để chọn làm tâm $\rightarrow n$ lựa chọn cho mỗi trong K cụm.
 - $K!$: Có $K!$ cách **hoán vị** (gán) K tâm đã chọn vào K cụm (vì thứ tự chọn tâm tương ứng với thứ tự cụm).
- **Mẫu số (Denominator):** Số cách chọn K tâm bất kỳ:
 - Có $Kn = N$ điểm để chọn.
 - Chọn **lần lượt** K tâm, mỗi lần có N lựa chọn $\rightarrow N^K = (Kn)^K$.
(Đây là chọn **có thứ tự**, vì thứ tự chọn tâm quan trọng trong tính toán này).
- Khi K tăng $\rightarrow P$ giảm rất nhanh $\rightarrow$ khởi tạo ngẫu nhiên trong K-Means dễ thất bại khi K lớn

▪ **Solutions (Giải pháp):**

- **Chạy nhiều lần (Multiple runs):** Chạy K-means nhiều lần với các khởi tạo ngẫu nhiên khác nhau và **chọn kết quả có SSE thấp nhất**. Đây là cách phổ biến và đơn giản nhất.
- **Chiến lược chọn tâm thông minh:** Sử dụng các thuật toán như **K-means++** để **chọn các tâm ban đầu cách xa nhau**, tăng khả năng có được kết quả tốt.
- **Sử dụng thuật toán khác: Bisecting K-means** ít nhạy cảm với khởi tạo hơn.

5.5.1.5. K-means++

- Thuật toán K-means tiêu chuẩn thường bắt đầu bằng cách chọn K tâm ban đầu một cách hoàn toàn ngẫu nhiên. Nếu chọn không may, các tâm có thể nằm gần nhau trong cùng một cụm, dẫn đến kết quả phân cụm rất tệ (kẹt ở cực tiểu địa phương).
- K-means++ là một thuật toán để **chọn các tâm ban đầu một cách thông minh hơn**. Nó không phải là một thuật toán phân cụm hoàn chỉnh.
- **To select a set of initial centroids, C, perform the following**
 - Chọn tâm đầu tiên **ngẫu nhiên**.
 - Với mỗi điểm còn lại, tính khoảng cách bình phương ($D(x)^2$) đến tâm **gần nhất** đã được chọn.
 - For each of the N points, x_i , $1 \leq i \leq N$, the minimum squared distance to the currently selected centroids, $C_1, \dots, C_j$, $1 \leq j < k$, $\min_j d^2(C_j, x_i)$
 - Chọn điểm tiếp theo làm tâm mới, với xác suất chọn tỷ lệ thuận với $D(x)^2$. Điều này có nghĩa là **các điểm ở xa các tâm đã chọn có khả năng được chọn cao hơn**.

$$\frac{\min_j d^2(C_j, x_i)}{\sum_i \min_j d^2(C_j, x_i)}$$
 - Lặp lại cho đến khi đủ K tâm.
- Cách này giúp dàn đều các tâm ban đầu ra khắp không gian dữ liệu.
- Khi bạn sử dụng K-means, bạn nên dùng K-means++ làm phương pháp khởi tạo mặc định thay vì khởi tạo ngẫu nhiên hoàn toàn. Hầu hết các thư viện học máy hiện đại (như Scikit-learn của Python) đều sử dụng K-means++ làm mặc định cho thuật toán K-means của họ.

5.5.1.6. K-medoids: PAM (Partitioning Around Medoids)

- Thay vì dùng điểm trung bình ảo, thuật toán này **chọn một điểm dữ liệu thực tế trong cụm làm đại diện (medoid)**. Mục tiêu là **tìm ra các medoid sao cho tổng khoảng cách từ các điểm đến medoid của chúng là nhỏ nhất**.

B1: Chọn ngẫu nhiên k đối tượng làm điểm trung tâm medoid ban đầu của k cụm

B2: Gán từng đối tượng vào cụm có medoid gần nó nhất (dựa trên khoảng cách Manhattan). Tính cost cho việc phân cụm.

$$Cost = \sum_{i=1}^k \sum_{x_j \in C_i} dist(x_j, c_i) \quad \begin{array}{l} \bullet x_j: \text{một điểm dữ liệu trong cụm } C_i \\ \bullet c_i: \text{medoid của cụm } C_i \\ \bullet dist(): \text{khoảng cách Manhattan} \end{array} \quad (5)$$

$$dist(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \dots + |x_{ip} - x_{jp}| \quad (2)$$

B3: Với mỗi medoid, chọn 1 đối tượng bất kỳ, hoán đổi nó với medoid của cụm.

- Nếu cost giảm so với trước đó thì quay lại B2.
- Ngược lại, undo lại bước hoán đổi thực hiện tiếp tục B3 cho đến khi không còn có thay đổi.
- The **time complexity** is $O(k * (n - k)^2)$.
- **Ưu điểm lớn nhất: Mạnh mẽ (robust)** trước các điểm ngoại lai. Điểm ngoại lai (như điểm (6,6) ở trên) sẽ không bao giờ được chọn làm medoid vì nó quá xa các điểm khác. Do đó, tâm cụm sẽ đại diện tốt hơn cho phần đông các điểm.
- **Ưu điểm khác:** Vì không cần tính "trung bình", K-medoids có thể hoạt động trên các loại dữ liệu phức tạp (ví dụ: dữ liệu văn bản, chuỗi DNA) miễn là bạn có thể định nghĩa một hàm khoảng cách/bất tương đồng giữa hai đối tượng bất kỳ.
- **Khi nào dừng?** Khi dữ liệu của bạn có nhiều nhiễu hoặc outliers, hoặc khi dữ liệu của bạn không phải là dạng số mà việc tính trung bình không có ý nghĩa.
- PAM hiệu quả với tập dữ liệu nhỏ nhưng không mở rộng tốt với tập dữ liệu lớn
- Cải tiến:
 - CLARA– Clustering LARge Applications (Kaufmann & Rousseeuw, 1990): dựa trên phương pháp lấy mẫu
 - CLARANS– Clustering LARge Applications based upon RANdomized Search (Ng&Han, 1994): lấy mẫu động

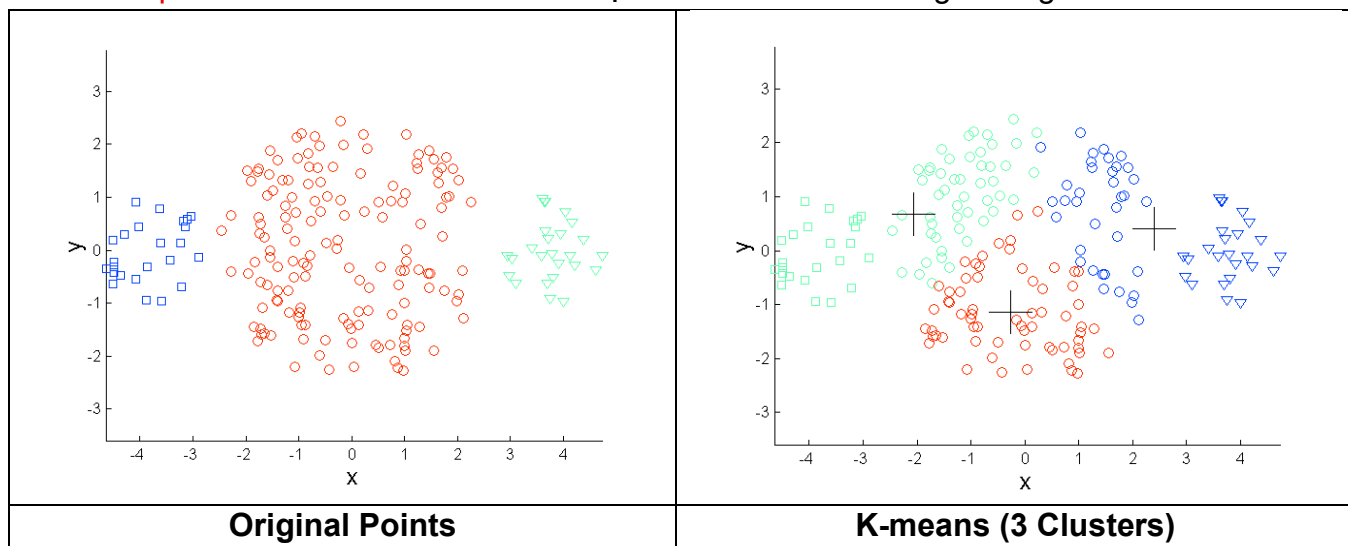
Tiêu chí	K-means	K-medoids	K-means++
Mục đích chính	Thuật toán phân cụm	Thuật toán phân cụm	Phương pháp khởi tạo cho K-means
Cách xác định tâm cụm	Centroid: Là điểm trung bình của tất cả các điểm trong cụm. (Có thể không phải là một điểm dữ liệu thực tế)	Medoid: Là điểm dữ liệu thực tế trong cụm có khoảng cách trung bình đến các điểm khác là nhỏ nhất.	Chọn các tâm ban đầu cho K-means một cách thông minh, cách xa nhau.
Độ nhạy với điểm ngoại lai (Outliers)	Rất nhạy cảm. Các điểm ngoại lai có thể kéo centroid ra xa tâm thực của cụm.	Mạnh mẽ (Robust). Vì medoid phải là một điểm thực tế nên nó ít bị ảnh hưởng bởi các điểm ngoại lai.	Cải thiện K-means, nhưng bản thân K-means sau khi khởi tạo vẫn nhạy cảm.
Yêu cầu về dữ liệu/Khoảng cách	Cần dữ liệu số để tính trung bình. Thường dùng khoảng cách Euclidean.	Rất linh hoạt. Có thể làm việc với bất kỳ độ đo bất tương đồng nào (dissimilarity matrix).	Giống K-means.
Độ phức tạp tính toán	Nhanh, hiệu quả về mặt tính toán.	Thường chậm hơn K-means đáng kể.	Thêm một chút chi phí lúc khởi tạo, nhưng thường giúp K-means hội tụ nhanh hơn.

5.5.1.7. Bisecting K-means

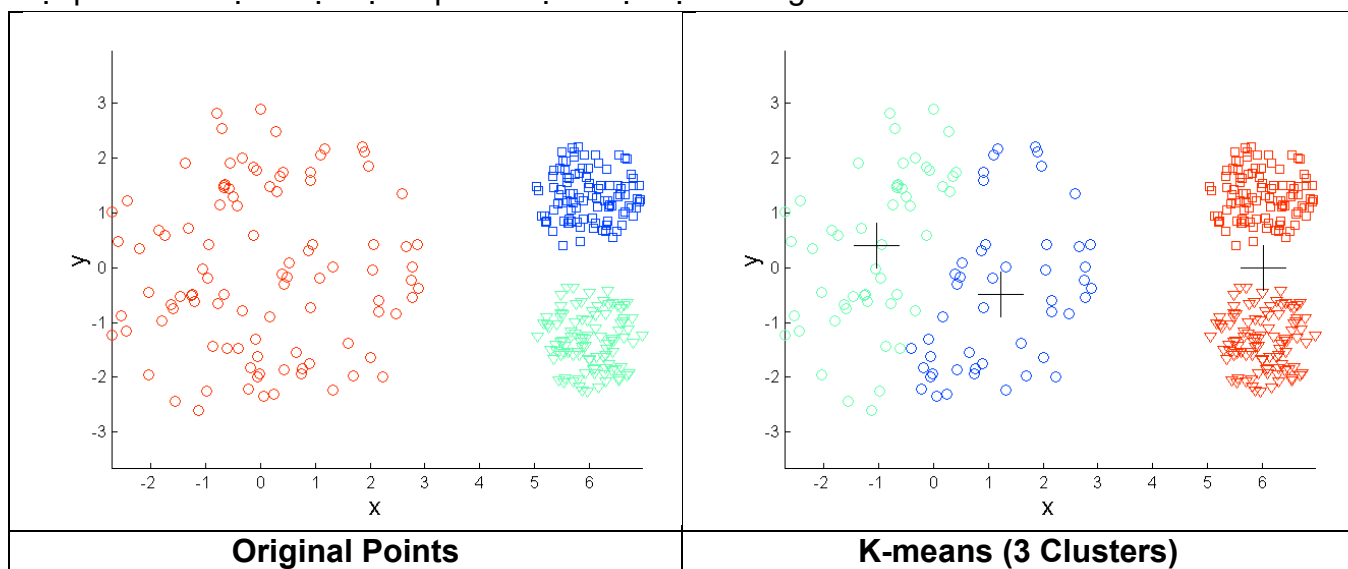
- Đây là một cách tiếp cận kết hợp giữa phân cấp và phân hoạch.
 - Bắt đầu với **1 cụm chứa tất cả các điểm**.
 - **Lặp lại:**
 - Chọn một cụm để tách (thường là cụm có SSE lớn nhất).
 - Dùng K-means để tách cụm đó thành hai cụm con.
 - Dừng lại khi có đủ K cụm.
- Phương pháp này ít bị ảnh hưởng bởi khởi tạo vì ở mỗi bước, việc tách đôi một cụm thường khá ổn định.

5.5.1.8. Limitations of K-means (Hạn chế của K-means)

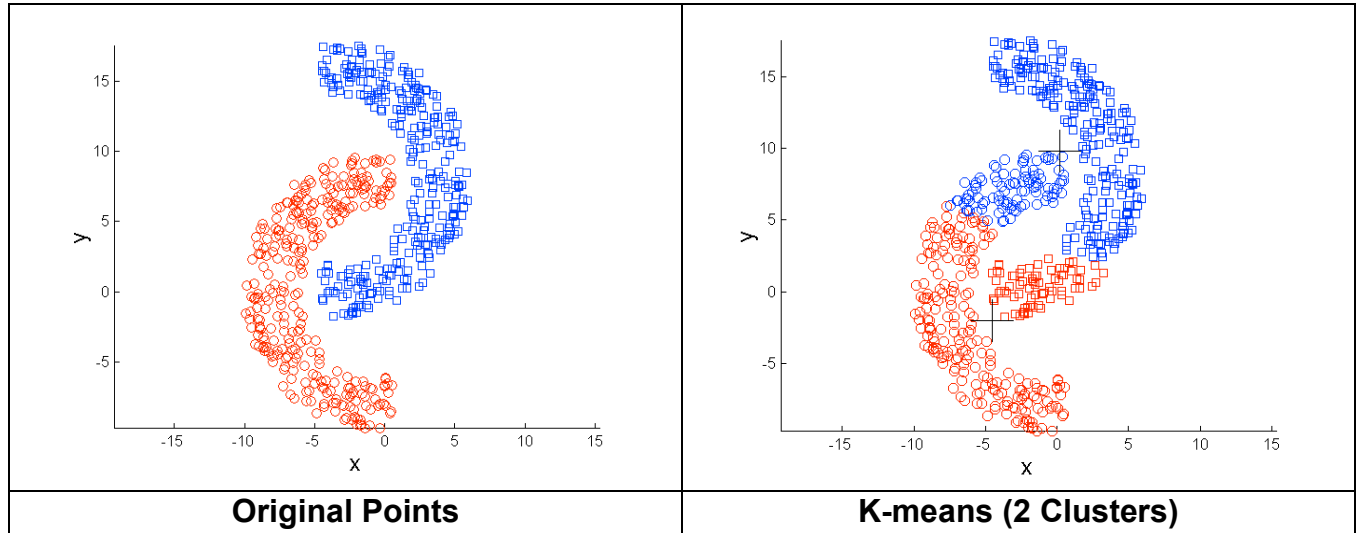
- **Differing Sizes:** Khi các cụm có kích thước khác nhau, **K-means có xu hướng chia cụm lớn thành các phần nhỏ hơn** để làm cho các cụm có kích thước tương đương.



- **Differing Density:** Khi các cụm có mật độ khác nhau, K-means gặp khó khăn và có thể gộp một phần của cụm mật độ thấp vào cụm mật độ cao ở gần nó.



- **Non-globular Shapes:** K-means hoàn toàn thất bại với **các cụm không có dạng hình cầu** (như hai hình xoắn ốc). Nó sẽ cắt ngang các hình dạng này một cách tùy tiện.

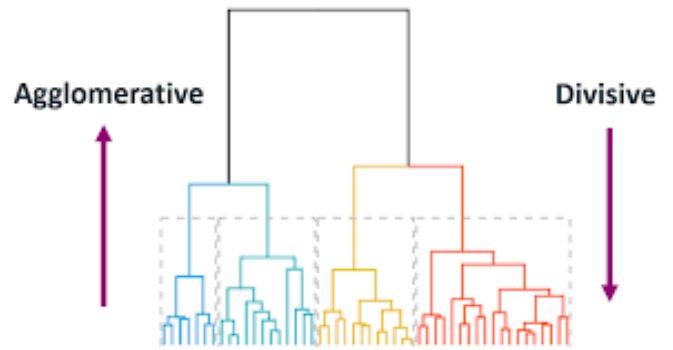


- **Overcoming Limitations:** Một giải pháp là **phân cụm dư (over-clustering)**. Ta chạy K-means với số K lớn hơn nhiều so với số cụm thực tế. Mỗi cụm nhỏ sẽ đại diện cho một phần của cụm lớn, phức tạp. Sau đó, ta thực hiện một bước hậu xử lý để gộp các cụm nhỏ này lại với nhau.
- Ngoài ra còn có những nhược điểm khác:
 - Chỉ áp dụng cho các đối tượng trong không gian n chiều liên tục. Thuộc tính không phải dạng số?
 - Sử dụng phương pháp k-mode cho dữ liệu danh mục
 - **k-medoids** có thể được áp dụng cho nhiều loại dữ liệu
 - Cần xác định trước số cụm k.
 - Phụ thuộc vào việc khởi tạo các cụm đầu tiên
 - Nhạy cảm với dữ liệu nhiễu, cá biệt.

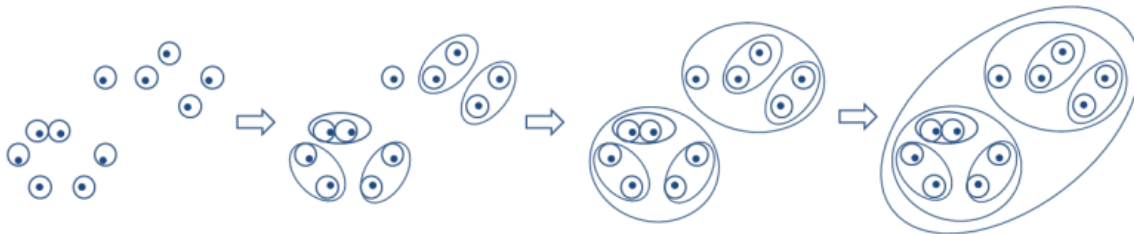
5.5.2. Hierarchical Clustering

- Tạo ra một cấu trúc cây các cụm lồng nhau (nested clusters), có thể được biểu diễn bằng **dendrogram**.
- **Strengths of Hierarchical Clustering**
 - **Không cần xác định trước số cụm K:** Bạn có thể quyết định số cụm sau khi đã có dendrogram bằng cách "cắt" cây ở độ cao phù hợp.
 - **Tương ứng với phân loại có ý nghĩa:** Cấu trúc cây rất tự nhiên trong nhiều lĩnh vực, ví dụ như cây phân loại sinh học (giới, ngành, lớp, bộ...).
- **Two main types of hierarchical clustering**
 - **Agglomerative (bottom-up):**
 - import từ sklearn.
 - **Bắt đầu với mỗi điểm là một cụm.**
 - Ở mỗi bước, gộp hai cụm gần nhất lại với nhau.

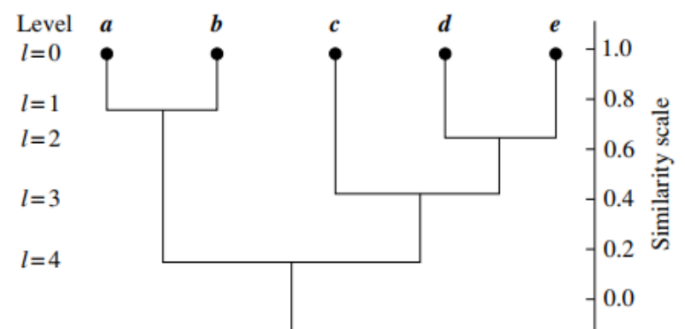
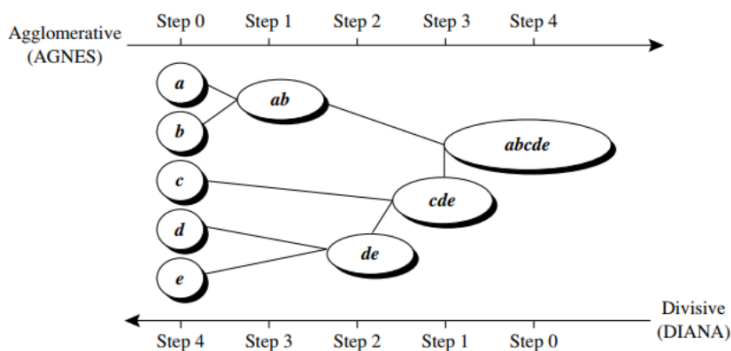
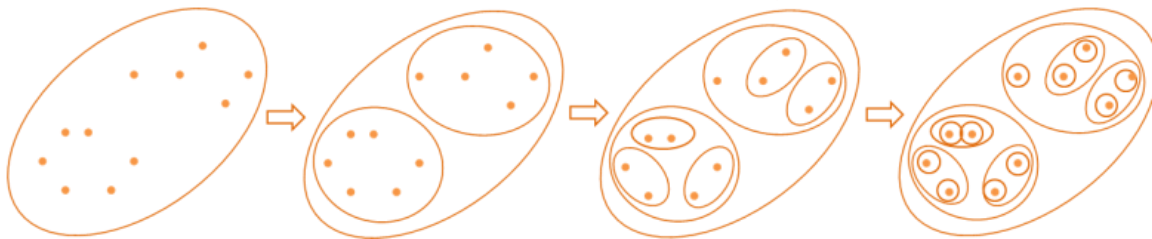
- Tiếp tục cho đến khi **chỉ còn một cụm (or k clusters)**.
- Đây là phương pháp phổ biến nhất.
- **Divisive (top-down):**
 - Bắt đầu với **một cụm chứa tất cả các điểm**.
 - Ở mỗi bước, **tách một cụm thành hai cụm con**.
 - Tiếp tục cho đến khi **mỗi điểm là một cụm riêng**.
 - **Phương pháp này ít phổ biến hơn vì việc "tách" một cụm là một bài toán khó.**
 - **Trong Python không có thư viện để ta import.**



Agglomerative Hierarchical Clustering



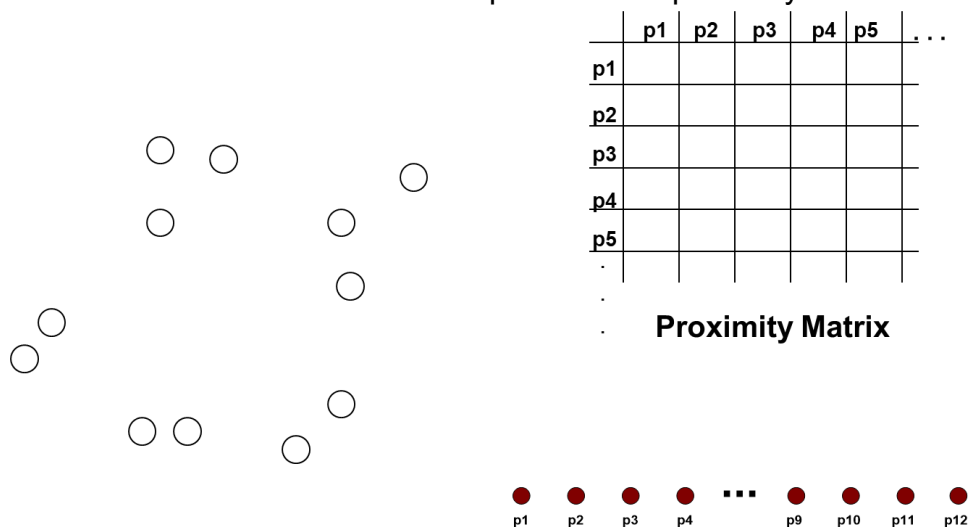
Divisive Hierarchical Clustering



- Traditional hierarchical algorithms use a **similarity or distance matrix**: Merge or split one cluster at a time.

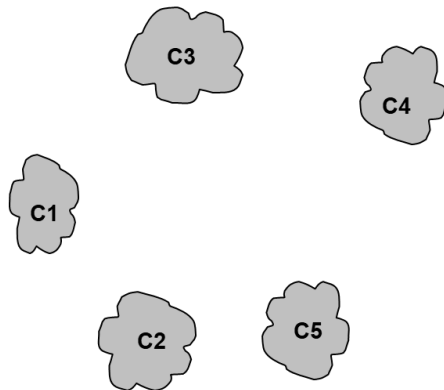
5.5.2.1. Agglomerative Clustering Algorithm (Thuật toán Tích tụ)/ AGNES (Agglomerative Nesting)

- **Key Idea: Successively merge closest clusters (Hợp nhất liên tiếp các cụm gần nhất)**
- **Thuật toán:**
 - 1/ Compute the **proximity matrix** // Ma trận khoảng cách ban đầu
 - Ma trận này là một bảng vuông, lưu trữ khoảng cách (hoặc độ tương đồng) giữa **mọi cặp điểm** trong dữ liệu. Nếu có N điểm, ma trận sẽ có kích thước N x N.
 - 2/ Let **each data point be a cluster** // n cụm, mỗi cụm 1 điểm
 - 3/ Repeat
 - 4/ Merge the two closest clusters // Hợp nhất 2 cụm gần nhất
 - Trong mỗi vòng lặp, thuật toán sẽ "quét" qua ma trận khoảng cách hiện tại để tìm ra cặp cụm có khoảng cách nhỏ nhất (hoặc độ tương đồng lớn nhất).
 - 5/ Update the proximity matrix // Cập nhật khoảng cách đến cụm mới
 - Sau khi gộp hai cụm (ví dụ C_i và C_j) thành một cụm mới (C_{new}), ta phải:
 - Xóa các hàng và cột tương ứng với C_i và C_j .
 - Thêm một hàng và một cột mới cho C_{new} .
 - Tính toán khoảng cách từ C_{new} đến tất cả các cụm còn lại.
 - 6/ Until only **a single cluster remains** // Hoặc đạt K cụm
 - Vòng lặp sẽ chạy N-1 (hoặc K-1) lần. Sau mỗi lần gộp, số lượng cụm giảm đi 1.
- **Steps 1 and 2: Start with clusters of individual points and a proximity matrix**



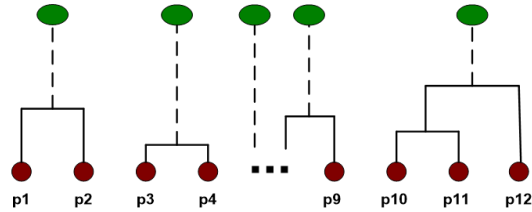
- **Ban đầu**, ma trận này sẽ có kích thước 12x12.
- Ô ở hàng p_i và cột p_j sẽ chứa giá trị khoảng cách giữa điểm p_i và điểm p_j .
- Đây chính là kết quả của **Bước 1** (Compute the proximity matrix).
- Việc coi mỗi điểm là một cụm là **Bước 2**.
- **Hình ảnh các điểm $p_1...p_{12}$ ở dưới:** Đây là sự khởi đầu của cây dendrogram. Các điểm này là các "lá" (leaves) của cây, là các đơn vị cơ bản nhất.

- After some merging steps (**sau một vài vòng lặp**), we have some clusters



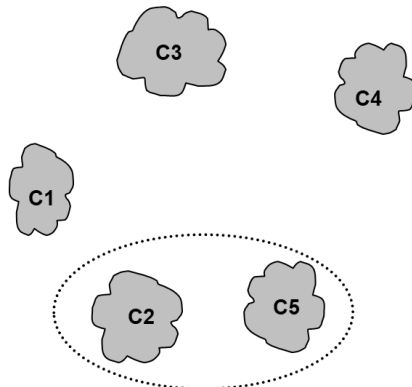
	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix



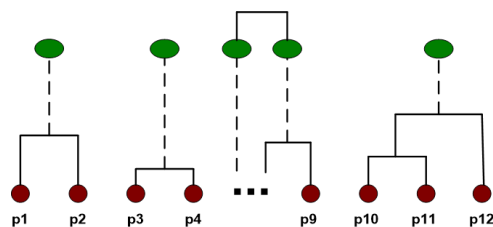
- Các điểm không còn đứng riêng lẻ nữa. Chúng đã được gộp lại thành 5 cụm lớn hơn: C1, C2, C3, C4, C5. Điều này có nghĩa là vòng lặp Repeat trong đã chạy $12 - 5 = 7$ lần.
- Ma trận khoảng cách đã thu nhỏ lại:** Ma trận bây giờ có kích thước 5×5 . Các hàng và cột không còn là các điểm p_i nữa, mà là các cụm C_i .
- Cây dendrogram đã được xây dựng một phần:** các "lá" ($p_1, p_2, \dots$) đã được gộp lại thành các nhánh cây. 5 chấm xanh ở trên cùng của các nhánh cây này tương ứng với 5 cụm C1 đến C5. Điều này cho thấy cấu trúc phân cấp đang dần được hình thành.

- Step 4:** We want to merge the two closest clusters (**C2 and C5**) and update the proximity matrix.



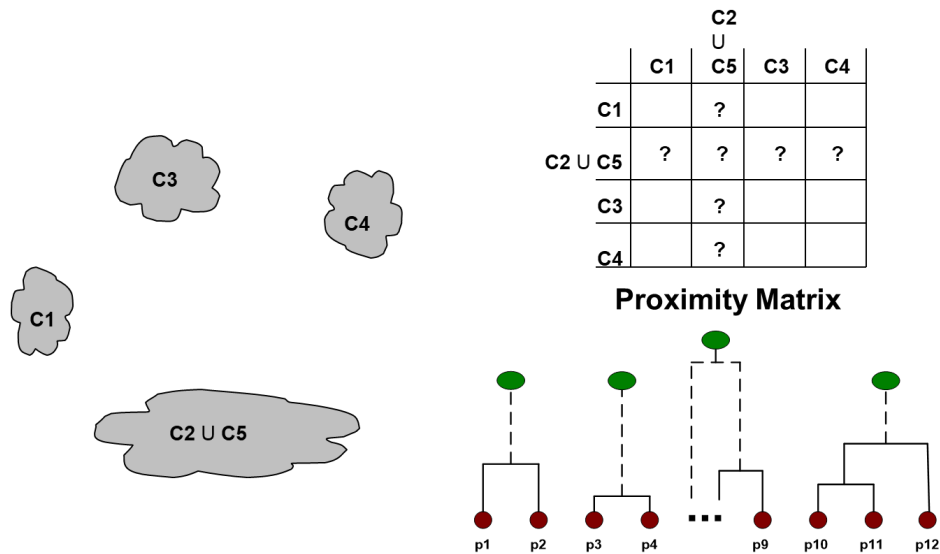
	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix



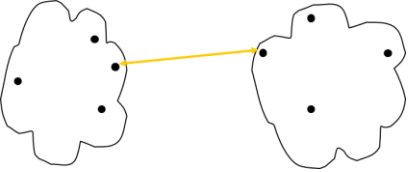
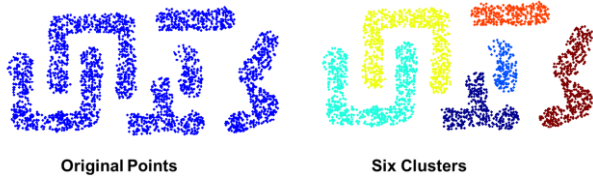
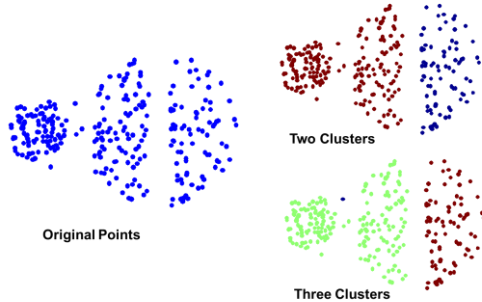
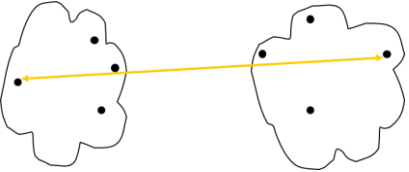
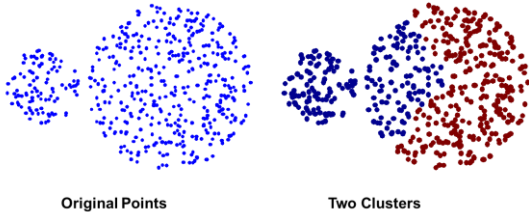
- Các hàng và cột tương ứng với C2 và C5 trong ma trận được bôi xám. Điều này ngụ ý rằng sau khi gộp, hai hàng và hai cột này sẽ bị loại bỏ.

- **Step 5:** The question is “How do we update the proximity matrix?”



- **Cây dendrogram:** Nhánh đại diện cho C2 và nhánh đại diện cho C5 đã được kết nối với nhau tại một nút cao hơn, biểu thị cho hành động gộp vừa diễn ra.
- Các dấu chấm hỏi (?) trong ma trận đại diện cho các giá trị mà chúng ta cần phải tính toán. Cụ thể:
 - $\text{dist}(C1, C2 \cup C5) = ?$
 - $\text{dist}(C3, C2 \cup C5) = ?$
 - $\text{dist}(C4, C2 \cup C5) = ?$

• **How to Define Inter-Cluster Distance (Cách định nghĩa Khoảng cách giữa các Cụm)**

	MIN (Single Link): Khoảng cách giữa hai cụm là khoảng cách giữa hai điểm gần nhất thuộc hai cụm đó. Ưu điểm: Có thể xử lý các cụm có hình dạng phi cầu, kéo dài (hiệu ứng chuỗi - chaining effect).  Original Points Six Clusters Nhược điểm: Rất nhạy cảm với nhiễu. Một vài điểm nhiễu nằm giữa hai cụm có thể "bắt cầu" và khiến hai cụm đó bị gộp lại sai.  Original Points Two Clusters Three Clusters Khi nào nên dùng? Khi bạn nghi ngờ dữ liệu của mình có các cụm hình dạng phức tạp, không phải hình cầu, và dữ liệu tương đối sạch.
	MAX (Complete Link): Khoảng cách giữa hai cụm là khoảng cách giữa hai điểm xa nhất thuộc hai cụm đó. Ưu điểm: Ít nhạy cảm với nhiễu hơn so với MIN. Tạo ra các cụm rất đặc và chặt chẽ. Nhược điểm: Có xu hướng phá vỡ các cụm lớn và ưu tiên các cụm có dạng hình cầu (globular).  Original Points Two Clusters Khi nào nên dùng? Khi mục tiêu là tìm các cụm nhỏ, đặc và được phân tách rõ ràng. Khi nhiễu là một vấn đề.

Group Average: Khoảng cách giữa hai cụm là **khoảng cách trung bình của tất cả các cặp điểm** (mỗi điểm từ một cụm).

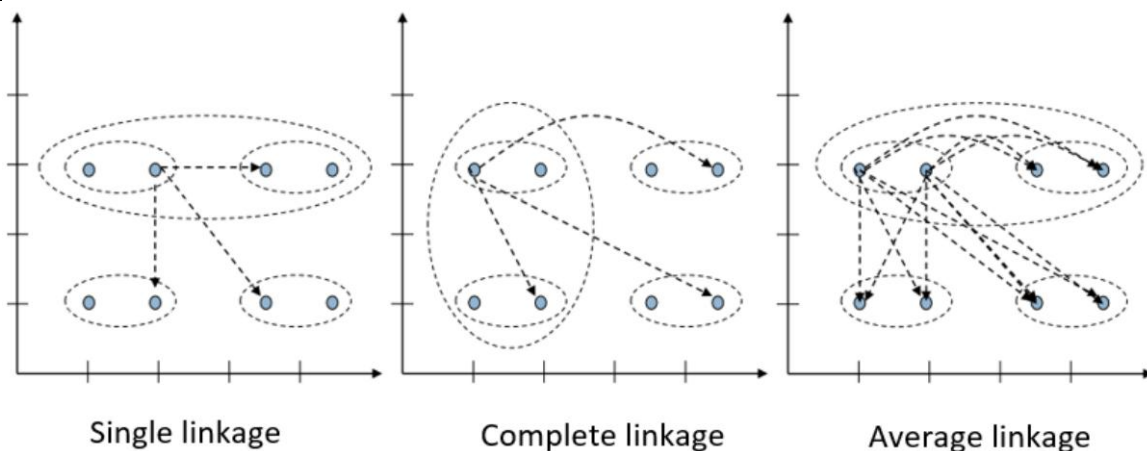
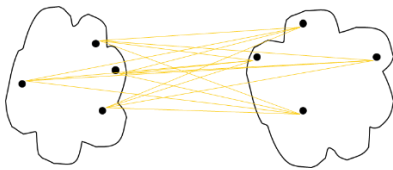
$$\text{proximity}(\text{Cluster}_i, \text{Cluster}_j) = \frac{\sum_{\substack{p_i \in \text{Cluster}_i \\ p_j \in \text{Cluster}_j}} \text{proximity}(p_i, p_j)}{|\text{Cluster}_i| \times |\text{Cluster}_j|}$$

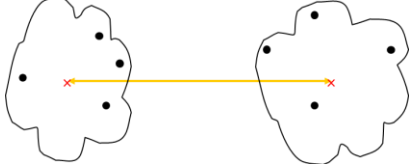
- **tử số:** Tính tổng khoảng cách của **TẤT CẢ** các cặp điểm (p_i, p_j) có thể có, trong đó điểm đầu tiên lấy từ Cluster_i và điểm thứ hai lấy từ Cluster_j .
- **mẫu số:** Đây chính là **tổng số cặp điểm** (p_i, p_j) mà bạn đã tính tổng ở tử số.
 - $|\text{Cluster}_i|$: Kích thước (số lượng điểm) của Cluster_i .
 - $|\text{Cluster}_j|$: Kích thước (số lượng điểm) của Cluster_j .

Đây được coi là một sự **thỏa hiệp** giữa MIN và MAX. Nó cân bằng hơn, ít bị ảnh hưởng bởi nhiễu hơn MIN nhưng vẫn có xu hướng ưu tiên các cụm hình cầu.

- **So với MIN (Single Link):** MIN chỉ dựa vào *một cặp điểm gần nhất*. Nó rất nhạy cảm với nhiễu, vì chỉ cần một điểm nhiễu "bắc cầu" là hai cụm có thể bị gộp sai. Group Average ổn định hơn nhiều vì nó xem xét **tất cả** các điểm, nên ảnh hưởng của một cặp điểm đơn lẻ sẽ bị giảm đi.
- **So với MAX (Complete Link):** MAX chỉ dựa vào *một cặp điểm xa nhất*. Nó có xu hướng phá vỡ các cụm lớn và ưu tiên các cụm hình cầu. Group Average ít cực đoan hơn vì nó lấy trung bình, không bị chi phối bởi một cặp điểm ở quá xa.

Khi nào nên dùng? Đây thường là **một lựa chọn mặc định tốt** khi bạn không chắc chắn. Nó cân bằng và hoạt động tốt trong nhiều trường hợp.



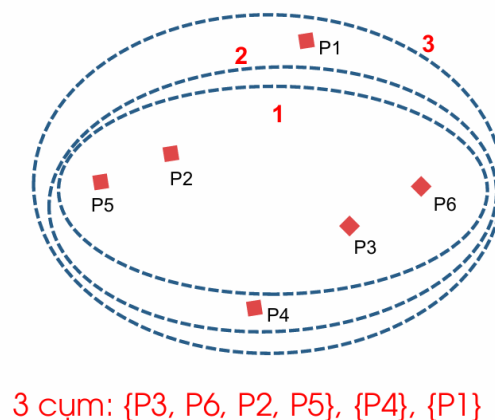
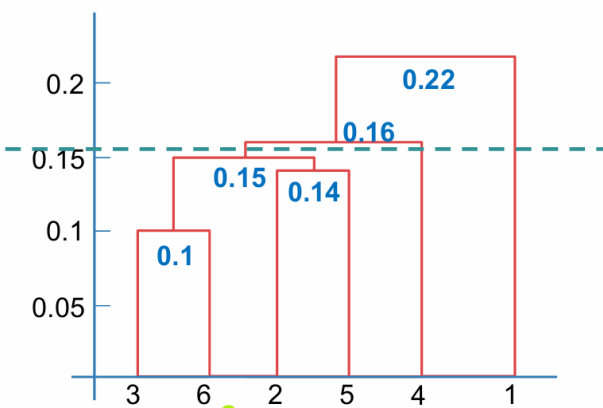
	Distance Between Centroids: Khoảng cách giữa tâm (centroid) của hai cụm. - Hiệu quả về mặt tính toán. - Có thể xảy ra hiện tượng đảo ngược (inversion), nơi khoảng cách gộp hai cụm có thể nhỏ hơn khoảng cách gộp ở bước trước, làm cho dendrogram khó diễn giải. (ít được sử dụng vì lý do này). Gần như không bao giờ là lựa chọn tốt nhất trong thực tế so với Group Average hay Ward's Method.
---	---

Other methods driven by an objective function → Ward's Method: Gộp hai cụm nào mà làm cho sự gia tăng của tổng bình phương sai số (SSE) là **nhỏ nhất**.

- Quá trình hoạt động của Ward's Method:** Hãy tưởng tượng bạn đang ở một bước trung gian và có các cụm C1, C2, C3, C4.
 - Thử nghiệm gộp:** Thuật toán sẽ "giả vờ" gộp từng cặp cụm và tính SSE mới:
 - Thử gộp C1 và C2:**
 - Tính SSE của cụm mới (C1 U C2).
 - Tính **độ tăng SSE = SSE(C1 U C2) - (SSE(C1) + SSE(C2))**. Đây là "chi phí" của việc gộp C1 và C2.
 - Thử gộp C1 và C3:** Tính chi phí tương tự.
 - ...
 - Làm điều này cho tất cả các cặp cụm có thể có.
 - Quyết định gộp:** Thuật toán sẽ tìm ra cặp cụm (ví dụ C_i và C_j) mà có **độ tăng SSE (chi phí) là nhỏ nhất**.
 - Thực hiện gộp:** Gộp C_i và C_j thành một cụm mới. Lặp lại quá trình cho đến khi chỉ còn một cụm.
- Less susceptible to noise (ít nhạy cảm với nhiễu)
- Biased towards globular clusters (thiên vị với các cụm hình cầu) (**giống K-means**)

Khi nào nên dừng? Khi bạn tin rằng các cụm trong dữ liệu của mình có dạng hình cầu và kích thước tương đối đồng đều. Rất phổ biến và thường cho kết quả tốt.

VD6: Chia 03 cụm



5.5.2.2. Divisive Clustering Algorithm (Thuật toán Phân cụm Phân chia)/ DIANA - Divisive ANALysis

- **Ý tưởng cốt lõi (Top-down):** Bắt đầu với một cụm duy nhất chứa tất cả các điểm dữ liệu. Ở mỗi bước, nó sẽ chọn một cụm và **tách** nó thành hai cụm con.
 - **Chọn cụm nào để tách?** Thường là cụm "lông lẻo" nhất, lớn nhất, hoặc có đường kính lớn nhất.
 - **Tách cụm đó như thế nào?** Đây là phần khó nhất. Làm thế nào để tìm ra cách chia một nhóm điểm thành hai nhóm con một cách tối ưu?
- **Thuật toán**
 1. **Bắt đầu:** Tất cả các điểm thuộc một cụm.
 2. **Lặp lại cho đến khi mỗi điểm là một cụm riêng:**
 - a. **Chọn cụm để tách:** **Tìm cụm có đường kính lớn nhất** (khoảng cách lớn nhất giữa hai điểm bất kỳ trong cụm).
 - b. **Bắt đầu quá trình tách cụm đó:**
 - i. Tìm điểm trong cụm này có **khoảng cách trung bình đến các điểm khác là lớn nhất**. Điểm này sẽ bắt đầu một "nhóm tách ra" (splinter group).
 - ii. **Lặp lại:** Với mỗi điểm chưa thuộc "nhóm tách ra", hãy tính xem nó **"gần" nhóm cũ hơn hay "nhóm tách ra" hơn**. Cụ thể, tính $D = (\text{khoảng cách trung bình đến các điểm trong nhóm cũ}) - (\text{khoảng cách trung bình đến các điểm trong nhóm tách ra})$.
 - iii. Nếu $D > 0$, điểm đó gần với "nhóm tách ra" hơn. **Tìm điểm có giá trị D lớn nhất** và chuyển nó sang "nhóm tách ra".
 - iv. Lặp lại bước ii và iii cho **đến khi tất cả các điểm đều có $D \leq 0$** (tức là không còn điểm nào muốn chuyển sang nhóm mới nữa).
 - c. Bây giờ cụm đã được tách thành hai cụm con.
 3. Chọn cụm nhỏ có đường kính lớn nhất. Sau đó chia cụm này, làm theo các bước từ đầu đến đây. Lặp lại cho đến khi tất cả các cụm chỉ chứa một đối tượng.
- **Tại sao nó ít phổ biến?**
 - **Độ phức tạp tính toán:** Việc xem xét tất cả các cách có thể để tách một cụm là một bài toán tổ hợp khổng lồ ($O(2^{N-1})$). Các phương pháp phỏng đoán (heuristic) như DIANA tuy nhanh hơn nhưng vẫn phức tạp.
 - **Agglomerative thường trực quan hơn:** Bắt đầu từ những thứ nhỏ nhất và xây dựng lên thường dễ hiểu và dễ thực hiện hơn.

5.5.2.3. Time and Space requirements, Problems and Limitations

- **Time and Space requirements**
 - $O(N^2)$ space since it uses the proximity matrix. (N is the number of points).
 - $O(N^3)$ time in many cases
 - There are N steps and at each step the size, N^2 , proximity matrix must be updated and searched
 - Complexity can be reduced to $O(N^2 \log(N))$ time with some cleverness

- **Problems and Limitations**

- Once a decision is made to combine two clusters, **it cannot be undone**. Điều này có thể dẫn đến các lựa chọn không tối ưu ở các bước sau.
- Khó xác định phương pháp tích tụ hay chia nhỏ
- **No global objective function** is directly **minimized**
- Different schemes have problems with one or more of the following:
 - Sensitivity to noise
 - Difficulty handling clusters of different sizes and non-globular shapes
 - Breaking large clusters
- Tích hợp phương pháp phân cấp với phân hoạch: BIRCH, CURE, CHAMELEON

5.5.3. Density-Based Clustering

- **Các cụm là những vùng có mật độ cao được ngăn cách với nhau bởi những vùng có mật độ thấp**. Thay vì định nghĩa cụm bằng một "tâm" (như K-means) hay bằng "khoảng cách" (như Hierarchical), nó định nghĩa cụm bằng "sự đông đúc".

	Đối với tập dữ liệu có hình dạng như hình bên: • K-means thất bại? K-means sẽ cố gắng tìm các "tâm" hình cầu. Nó sẽ cắt ngang các hình dạng phức tạp này một cách tùy tiện, không thể nào nhận ra các cấu trúc chữ U hay chữ S. • Hierarchical (MAX/Ward's) thất bại? Các phương pháp này cũng có xu hướng tìm các cụm hình cầu, đặc. Chúng sẽ chia nhỏ các hình dạng này. • Hierarchical (MIN) có thể hoạt động? MIN có thể tìm ra các cụm hình chuỗi, nhưng nó lại rất nhạy cảm với nhiễu. Chỉ cần một vài điểm nhiễu nằm giữa hai hình là chúng có thể bị gộp lại với nhau.
--	--

- Chúng ta cần một thuật toán mới có khả năng:
 - Nhận diện các cụm có hình dạng tùy ý.
 - Phân biệt được các vùng "đông đúc" (cụm) và các vùng "thưa thớt" (không gian ngăn cách hoặc nhiễu).

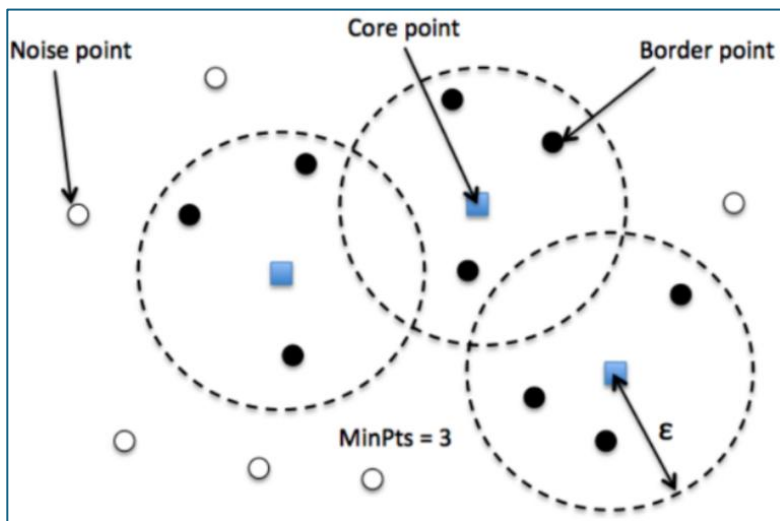
5.5.3.1. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

- DBSCAN is a density-based algorithm.
- Nó có thể phát hiện ra các cụm có hình dạng và kích thước khác nhau từ một lượng lớn dữ liệu chứa *nhiều*.

- **Các khái niệm:**

- **Hai tham số đầu vào:** DBSCAN yêu cầu người dùng cung cấp 2 tham số:
 - **Eps (ϵ -epsilon):** Một giá trị khoảng cách, dùng để định nghĩa một "vùng lân cận" (neighborhood). Nó giống như việc bạn vẽ **một vòng tròn có bán kính Eps** xung quanh một điểm.
 - **Density** = number of points within a specified radius (Eps)

- Nếu eps quá nhỏ, nhiều điểm trở thành nhiễu; nếu quá lớn, các cụm có thể gộp lại thành một.
- **MinPts**: Số lượng điểm **tối thiểu phải có bên trong vùng lân cận Eps** của một điểm để nó được coi là "đông đúc".
 - MinPts nhỏ (ví dụ, 2 hoặc 3) có thể dẫn đến các cụm nhỏ và nhiều noise.
 - MinPts lớn yêu cầu mật độ cao hơn, dẫn đến ít cụm hơn nhưng ổn định hơn.
- *eps kiểm soát kích thước vùng mật độ, min_pts xác định mức độ "đông đúc" cần thiết để hình thành cụm. Cả hai cần được chọn dựa trên phân bố dữ liệu để phân cụm hiệu quả.*
- **Phân loại điểm**: Dựa trên 2 tham số trên, **mỗi điểm trong dữ liệu sẽ được gán vào 1 trong 3 loại sau**:
 - **Core point (Điểm lõi)**: Đây là trái tim của một cụm. **Một điểm là điểm lõi nếu nó "đủ đông đúc", tức là có ít nhất MinPts điểm (tính cả chính nó) nằm trong bán kính Eps của nó.** (Trung tâm của một thành phố đông đúc).



"đủ đông đúc", tức là **có ít nhất MinPts điểm (tính cả chính nó) nằm trong bán kính Eps** của nó. (Trung tâm của một thành phố đông đúc).

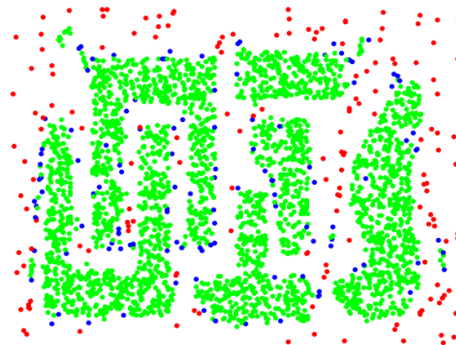
- **Border point (Điểm biên)**: Đây là rìa của một cụm. Một điểm là điểm biên **nếu nó không đủ đông đúc để làm điểm lõi, nhưng nó là "hàng xóm" của một điểm lõi** (tức là nó nằm trong bán kính Eps của một điểm lõi nào đó). (Vùng ngoại ô, không đông đúc bằng trung tâm nhưng vẫn thuộc về thành phố).

- **Noise point (Điểm nhiễu / Outlier)**: Một điểm không phải là điểm lõi và cũng không phải là điểm biên. Nó nằm một mình ở vùng thưa thớt. (Một căn nhà hẻo lánh giữa đồng không mông quạnh).

- **Ex: Eps = 10, MinPts = 4**



Original Points



Point types: **core**,
border and **noise**

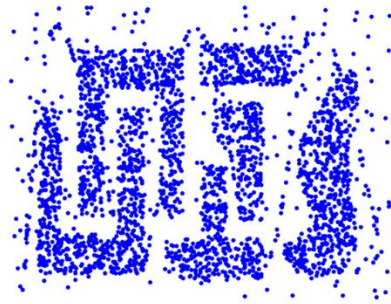
5.5.3.2. DBSCAN Algorithm

- **Input:** Dataset (các điểm), eps, MinPts
- **Output:** Nhãn cụm cho mỗi điểm (Cụm ID hoặc Noise)
- Thuật toán có thể được tóm tắt bằng hai ý tưởng chính: **Mở rộng (Expansion)** và **Kết nối (Connectivity)**.
 1. **Bước 1: Gán nhãn:** Thuật toán duyệt qua tất cả các điểm và gán nhãn cho chúng là "lỗi", "biên", hay "nhiều" dựa trên Eps và MinPts.
 - Với mỗi điểm x , đếm số điểm trong **Eps-neighborhood** của nó (gọi là $N(x)$).
 - Nếu $|N(x)| \geq \text{MinPts} \rightarrow$ **core point**.
 - Nếu $|N(x)| < \text{MinPts}$ nhưng **thuộc vùng Eps của 1 core point** $\rightarrow$ **border point**. Ngược lại $\rightarrow$ **noise point**.
 2. **Bước 2: Xây dựng cụm:**
 - **Loại bỏ nhiễu:** Tất cả các điểm được gán nhãn "nhiều" (màu đỏ) sẽ bị bỏ qua. Đây là một điểm mạnh lớn của DBSCAN.
 - **Kết nối các điểm lỗi:** Thuật toán hoạt động dựa trên khái niệm "density-reachability" (có thể đến được qua mật độ).
 - **Quy tắc 1 (Kết nối trực tiếp):** Nếu hai điểm lỗi nằm trong bán kính Eps của nhau, chúng thuộc cùng một cụm.
 - **Quy tắc 2 (Kết nối gián tiếp - Tính bắc cầu):** Nếu điểm lỗi A và B thuộc cùng cụm, và điểm lỗi B và C thuộc cùng cụm, thì A và C cũng thuộc cùng một cụm.
 - **Quá trình mở rộng:** Thuật toán sẽ bắt đầu từ một điểm lỗi bất kỳ, tìm tất cả các điểm lỗi khác có thể "kết nối" với nó (trực tiếp hoặc gián tiếp), và gom tất cả chúng vào một cụm. Quá trình này tạo ra "bộ xương" của các cụm.
 - **Gán các điểm biên:** Cuối cùng, thuật toán duyệt qua các điểm biên. Mỗi điểm biên được gán vào cụm của điểm lỗi mà nó là "hàng xóm".
 - Mỗi **border point** sẽ được gán vào cụm của **core point gần nhất** (hoặc một trong các cụm mà nó tiếp giáp).
 - Thực tế, nếu một border point nằm gần nhiều core points của các cụm khác nhau, DBSCAN sẽ gán nó **ngẫu nhiên vào một cụm bất kỳ trong số đó** (không có quy tắc duy nhất).
 - **Sau bước này, toàn bộ core + border points đã được gán cụm.**
- **Ví dụ:** Let's use a tiny 2D dataset to illustrate DBSCAN:
 - Dataset $X = [(1, 1), (1.5, 1), (2, 1), (5, 5), (5.5, 5), (10, 10)]$, so $n = 6, d = 2$.
 - Parameters: **eps = 1, minPts = 3**.
 - **Step 1: Initialize:** All points are unvisited, labels = $[-1, -1, -1, -1, -1, -1]$, $C = 0$.
 - **Step 2: Process First Point (1, 1)**
 - Mark (1,1) as visited.
 - **Neighbors within eps = 1:** (1.5,1), (2,1) (distance ≤ 1).
 - **Number of neighbors = 3** (including (1,1)), which is $\geq \text{minPts}$.
 - **(1, 1) is a core point.** Start a new cluster: $C = 1$.
 - **Assign (1,1) to cluster 1 and expand the cluster using its neighbors (1.5,1), (2,1).**

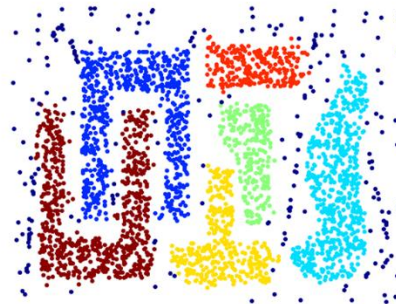
- Hoàn tất kết nối tất cả điểm trong Eps của (1,1) → (1.5,1) và (2,1).
 - Tiến hành Mở rộng cụm bằng cách kiểm tra các điểm láng giềng mới tìm được.
 - **Step 3: Expand Cluster 1**
 - **Point (1.5, 1):**
 - Mark (1.5,1) as visited.
 - Neighbors: (1,1), (2,1) (distance ≤ 1).
 - Number of neighbors = 3, so (1.5,1) is a core point.
 - Assign (1.5,1) to cluster 1.
 - Neighbors (1,1), (2,1) are already in the queue.
 - **Point (2, 1):**
 - Mark (2,1) as visited.
 - Neighbors: (1,1), (1.5,1).
 - Number of neighbors = 3, so (2,1) is a core point.
 - Assign (2,1) to cluster 1.
 - → Tất cả 3 core points đều trong cùng cụm.
 - Cluster 1 is now (1,1), (1.5,1), (2,1). Labels: [1,1,1, -1, -1, -1].
 - **Step 4: Process Next Unvisited Point (5, 5)**
 - Mark (5,5) as visited.
 - Neighbors: (5.5,5) (distance ≤ 1).
 - Number of neighbors = 2 (including (5,5)), which is $< \text{minPts}$.
 - Label (5,5) as noise (-1). Labels: [1,1,1, -1, -1, -1].
 - **Step 5: Process Next Unvisited Point (5.5, 5)**
 - Mark (5.5,5) as visited.
 - Neighbors: (5,5).
 - Number of neighbors = 2, which is $< \text{minPts}$.
 - Label (5.5,5) as noise (-1). Labels: [1,1,1, -1, -1, -1].
 - **Step 6: Process Next Unvisited Point (10, 10)**
 - Mark (10,10) as visited.
 - Neighbors: None within $\epsilon = 1$.
 - Number of neighbors = 1, which is $< \text{minPts}$.
 - Label (10,10) as noise (-1). Labels: [1,1,1, -1, -1, -1].
 - **Step 7: Output**
 - Final labels: [1,1,1, -1, -1, -1].
 - **Cluster 1:** (1, 1), (1.5, 1), (2, 1).
 - Noise points: (5,5), (5.5,5), (10,10).
- Trong DBSCAN, sau khi chạy thuật toán, các nhãn (labels) của các điểm dữ liệu sẽ được tự động tạo ra. Cụ thể:
 - Các điểm thuộc cụm sẽ được gán nhãn là các số không âm: 0, 1, 2, ... (mỗi số đại diện cho một cụm riêng biệt).
 - Các điểm nhiễu (noise/outliers) sẽ được gán nhãn -1.

5.5.3.2. Ưu và Nhược điểm của DBSCAN

- Khi hoạt động tốt:

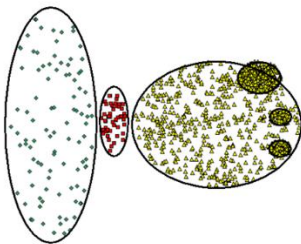


Original Points

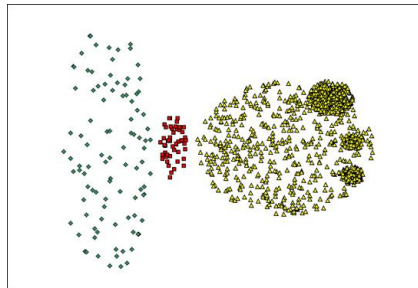


Clusters (dark blue points indicate noise)

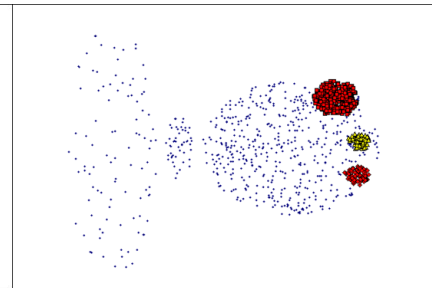
- DBSCAN thành công trên bộ dữ liệu phức tạp (**Can handle clusters of different shapes and sizes**). Nó đã nhận diện chính xác các cụm có hình dạng tùy ý và tách biệt chúng.
- Các điểm nhiễu được xác định và loại bỏ, không làm ảnh hưởng đến hình dạng của các cụm → **Resistant to noise (kháng nhiễu)**.
- Khi hoạt động không tốt:
 - Độ phức tạp cao đối với dữ liệu nhiều chiều.
 - Phụ thuộc vào giá trị Eps, MinPts.
 - **Vấn đề lớn nhất: Các cụm có mật độ khác nhau.**



Original Points



(MinPts=4, Eps=9.92).

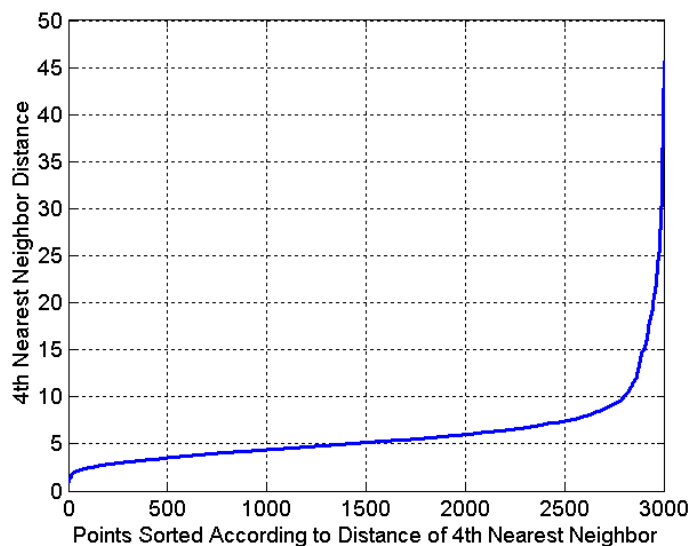


(MinPts=4, Eps=9.75)

- Hình ảnh cho thấy một bộ dữ liệu có một cụm thưa thớt bên trái và một cụm rất đông đúc bên phải.
- DBSCAN sử dụng một cặp tham số (Eps, MinPts) **toàn cục** cho toàn bộ dữ liệu. Đây là điểm yếu chí mạng của nó trong trường hợp này.
- **Kịch bản 1 (Eps lớn, Eps=9.92):** Để có thể nhận diện cụm thưa thớt bên trái, Eps phải đủ lớn. Nhưng với Eps lớn như vậy, **tất cả các cụm con dày đặc bên phải sẽ bị gộp chung thành một cụm duy nhất. Thuật toán mất đi khả năng nhìn thấy cấu trúc chi tiết.**
- **Kịch bản 2 (Eps nhỏ, Eps=9.75):** Để có thể tách các cụm con dày đặc bên phải, Eps phải nhỏ. Nhưng **với Eps nhỏ như vậy, các điểm trong cụm thưa thớt bên trái sẽ không còn "đủ đông đúc" nữa và bị coi là nhiễu.**

5.5.3.3. Determining EPS and MinPts (Cách chọn tham số)

- Cung cấp một phương pháp thực tế (heuristic) để chọn tham số **Eps** một cách có cơ sở, thay vì đoán mò.
- **Nguyên tắc:** Các điểm trong một cụm (lõi và biên) sẽ có các điểm lân cận ở gần. Ngược lại, các điểm nhiễu sẽ có các điểm lân cận ở rất xa. Do đó, nếu ta sắp xếp các điểm theo khoảng cách đến lân cận của chúng, sẽ có một "bước nhảy vọt" tại điểm ranh giới giữa điểm trong cụm và điểm nhiễu.
- Phương pháp k-distance plot (Biểu đồ khoảng cách thứ k):
 - Plot sorted distance of every point to its k^{th} nearest neighbor (sắp xếp khoảng cách của mỗi điểm đến điểm hàng xóm gần nhất thứ k của nó)
 - **Chọn MinPts (hay k):** Đây là một quy tắc kinh nghiệm. Một lựa chọn phổ biến là **MinPts = 2 * số chiều**. Với dữ liệu 2D, MinPts = 4 là một khởi đầu tốt.
 - **Tính khoảng cách:** Với $k = 4$, thuật toán sẽ tính cho mỗi điểm trong dữ liệu khoảng cách đến điểm láng giềng gần thứ 4 của nó.
 - **Vẽ biểu đồ:** Sắp xếp các khoảng cách này theo thứ tự từ nhỏ đến lớn và vẽ lên đồ thị.



- **Tìm "điểm khuỷu tay" (Elbow point):** Nhìn vào biểu đồ, bạn sẽ thấy một đoạn đồ thị gần như bằng phẳng hoặc tăng rất chậm, sau đó đột ngột dốc lên. Điểm mà đồ thị bắt đầu "bẻ cong" và dốc lên chính là "điểm khuỷu tay".
- **Chọn Eps:** Giá trị khoảng cách (trục Y) tại điểm khuỷu tay này là một lựa chọn tốt cho Eps. Nó đại diện cho ngưỡng tự nhiên phân tách các điểm "gần" (trong cụm) và các điểm "xa" (nhiều).

Thuật toán	Khi nào nên sử dụng?
K-means	1. Khi bạn cần tốc độ và hiệu quả: Đây là thuật toán phân cụm nhanh nhất, rất phù hợp cho các tập dữ liệu lớn (Big Data). 2. Khi bạn biết (hoặc có thể ước tính tốt) số lượng cụm (K): Thuật toán yêu cầu bạn phải cung cấp giá trị K từ trước. 3. Khi bạn tin rằng các cụm có dạng hình cầu (globular), kích thước tương đối bằng nhau và mật độ đồng đều: Đây là giả định ngầm của K-means. Nó hoạt động tốt nhất trong điều kiện lý tưởng này. 4. Khi dữ liệu của bạn tương đối "sạch", ít nhiễu và outliers: K-means rất nhạy cảm với các điểm ngoại lai có thể làm sai lệch vị trí của tâm cụm (centroid). 5. Khi mục tiêu là phân hoạch dữ liệu: Tức là mỗi điểm chỉ thuộc về một cụm duy nhất.
K-medoids	1. Khi dữ liệu của bạn có nhiều nhiễu hoặc outliers: Đây là ưu điểm lớn nhất. Vì tâm cụm (medoid) là một điểm dữ liệu thực tế, nó mạnh mẽ (robust) hơn nhiều so với centroid của K-means. 2. Khi việc tính "trung bình" không có ý nghĩa: K-medoids có thể hoạt động với bất kỳ loại dữ liệu nào (văn bản, chuỗi DNA,...) miễn là bạn có thể định nghĩa được một hàm khoảng cách/bất tương đồng giữa hai đối tượng. 3. Khi bạn muốn tâm cụm là một điểm đại diện thực tế, có thể diễn giải được: Ví dụ, tìm ra "khách hàng tiêu biểu nhất" cho mỗi phân khúc. 4. Khi tập dữ liệu không quá lớn: K-medoids có độ phức tạp tính toán cao hơn K-means, nên có thể chạy chậm trên các bộ dữ liệu rất lớn.
Agglomerative Hierarchical	1. Khi bạn không biết trước số lượng cụm: Thuật toán tạo ra một cây dendrogram, cho phép bạn chọn số lượng cụm sau khi đã xem xét cấu trúc phân cấp. 2. Khi bạn muốn khám phá mối quan hệ phân cấp, lồng nhau giữa các cụm: Rất hữu ích trong các lĩnh vực như sinh học (phân loại loài), xã hội học (cấu trúc tổ chức). 3. Khi bạn muốn có sự linh hoạt trong việc định nghĩa "khoảng cách cụm": Bạn có thể chọn MIN, MAX, Ward's, Group Average,... tùy thuộc vào đặc điểm dữ liệu (ví dụ, dùng MIN cho các cụm hình dạng kéo dài). 4. Khi tập dữ liệu có kích thước nhỏ đến trung bình: Độ phức tạp về thời gian ($O(N^3)$) và không gian ($O(N^2)$) làm cho nó không khả thi cho các tập dữ liệu rất lớn.
Divisive Hierarchical	1. Khi bạn quan tâm đến cấu trúc ở cấp độ vĩ mô hơn: Phương pháp "top-down" này tập trung vào việc tách các nhóm lớn trước, nên có thể hiệu quả hơn trong việc tìm ra các phân tách chính trong dữ liệu. 2. Khi bạn muốn một cách tiếp cận phân cấp khác: Đây là một phương pháp thay thế cho Agglomerative, mặc dù ít phổ biến hơn. 3. Trong các ứng dụng chuyên biệt: Divisive ít được sử dụng trong thực tế chung do độ phức tạp tính toán cao và có ít lựa chọn triển khai sẵn có hơn so với Agglomerative.

DBSCAN	1. Khi bạn nghi ngờ các cụm có hình dạng tùy ý, phức tạp (không phải hình cầu): Đây là thế mạnh vượt trội của DBSCAN. Nó có thể tìm thấy các cụm hình chữ S, hình chiếc nhẫn, v.v. 2. Khi dữ liệu của bạn có nhiều và bạn muốn tự động phát hiện và loại bỏ chúng: DBSCAN là một trong số ít các thuật toán có khả năng xác định rõ ràng các điểm outliers. 3. Khi bạn không muốn xác định trước số lượng cụm: DBSCAN tự động tìm ra số lượng cụm dựa trên mật độ dữ liệu. 4. Khi các cụm có mật độ tương đối đồng đều: Đây là điểm yếu của DBSCAN. Nó gặp khó khăn khi các cụm có mật độ rất khác nhau vì nó sử dụng một cặp tham số (Eps, MinPts) toàn cục.
---------------	---

5.6. Cluster Validity (Đánh giá Cụm)

- For **supervised classification** (*phân loại có giám sát*) we have a variety of measures to evaluate how good our model is **Accuracy, precision, recall**
- For cluster analysis, the analogous question is **how to evaluate the “goodness” of the resulting clusters?**
- **"Clusters are in the eye of the beholder"**. Sự phân cụm có thể mang tính chủ quan. Một cách phân cụm có thể tốt cho mục đích này nhưng không tốt cho mục đích khác.
- **Tại sao phải đánh giá?** Dù mang tính chủ quan, việc đánh giá là cực kỳ cần thiết để:
 - **Tránh tìm ra các mẫu trong nhiễu:** Các thuật toán có thể "ảo tưởng" ra các cụm ngay cả trong dữ liệu ngẫu nhiên.
 - **So sánh các thuật toán:** Thuật toán nào (K-means, DBSCAN,...) hoạt động tốt hơn trên dữ liệu của bạn?
 - **So sánh hai kết quả phân cụm:** Với K-means, kết quả với K=3 tốt hơn hay K=5 tốt hơn?
 - **So sánh hai cụm:** Cụm A có "chất lượng" tốt hơn cụm B không?
- **Các thuật toán phân cụm luôn đưa ra kết quả, ngay cả khi dữ liệu không có cấu trúc thực sự.**
 - **Random Points:** Dữ liệu được tạo ra một cách hoàn toàn ngẫu nhiên, không có bất kỳ nhóm tự nhiên nào.
 - **K-means:** Theo bản chất, K-means *bắt buộc* phải chia dữ liệu thành K phần. Nó sẽ tạo ra các vùng (gọi là Voronoi tessellation) và gọi chúng là các cụm, dù chúng vô nghĩa.
 - **DBSCAN & Complete Link:** Các thuật toán khác cũng sẽ cố gắng áp đặt một cấu trúc nào đó lên dữ liệu ngẫu nhiên.
 - **Đừng bao giờ tin tưởng một cách mù quáng vào kết quả mà thuật toán trả về. Bạn cần phải có phương pháp để xác thực xem các cụm đó có thực sự tồn tại hay không.**

5.6.1. Measures of Cluster Validity (Các độ đo Đánh giá Cụm)

- **Numerical measures** that are applied to judge various aspects of cluster validity, are classified into the following two types.
 - **Supervised (Có giám sát) / External Indices (Chỉ số ngoại sinh):**
 - Used to **measure** the extent to which **cluster labels match** externally supplied **class labels**.
 - **Khi nào dùng?** Khi bạn có "nhãn thật" (ground truth) của dữ liệu để so sánh. Ví dụ, bạn phân cụm hình ảnh các con vật, và bạn đã có sẵn nhãn "chó", "mèo", "chim".
 - **Ví dụ:** Entropy, Purity.
 - **Unsupervised (Không giám sát) / Internal Indices (Chỉ số nội sinh):**
 - **Khi nào dùng?** Đây là trường hợp phổ biến nhất, khi bạn không có nhãn thật và đang muốn khám phá cấu trúc của dữ liệu.
 - **Mục tiêu:** Đánh giá chất lượng của các cụm chỉ dựa trên thông tin bên trong dữ liệu (ví dụ: khoảng cách giữa các điểm).
 - **Ví dụ:** SSE, Silhouette Coefficient.
- You can use supervised or unsupervised measures to compare clusters or clusterings

5.6.1.1. Unsupervised Measures: Cohesion and Separation

- **Cluster Cohesion (Độ cố kết của cụm):** Đo lường mức độ "chặt chẽ", "gắn gũi" của các đối tượng trong cùng một cụm. **Một cụm tốt có độ cố kết cao.** **SSE (Sum of Squared Error)** là một ví dụ: SSE càng thấp, độ cố kết càng cao.

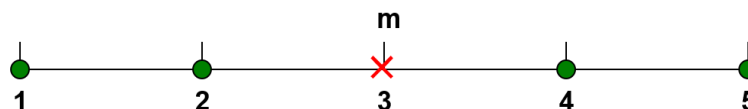
$$SSE = \sum_i \sum_{x \in C_i} (x - m_i)^2$$

- **Cluster Separation (Độ phân tách của cụm):** Đo lường mức độ "khác biệt", "tách rời" giữa một cụm và các cụm khác. **Các cụm tốt có độ phân tách cao.** **SSB (Between-cluster sum of squares)** là một ví dụ: SSB càng cao, độ phân tách càng lớn.

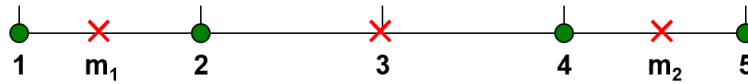
$$SSB = \sum_i |C_i| (m - m_i)^2$$

- Where $|C_i|$ is the size of cluster i

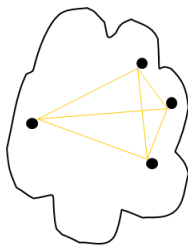
- **Example:**



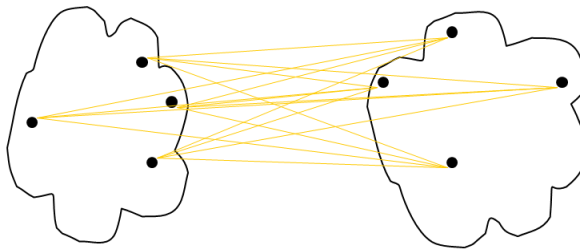
- **K=1**, tất cả các điểm thuộc một cụm.
 - $SSE = (1 - 3)^2 + (2 - 3)^2 + (4 - 3)^2 + (5 - 3)^2 = 10$
 - $SSB = 4 \times (3 - 3)^2 = 0$
 - $Total = 10 + 0 = 10$
- Cohesion rất thấp (SSE=10), và Separation bằng 0 (vì không có cụm nào khác để so sánh).



- **K=2**, các điểm được chia thành hai cụm.
 - $SSE = (1 - 1.5)^2 + (2 - 1.5)^2 + (4 - 4.5)^2 + (5 - 4.5)^2 = 1$
 - $SSB = 2 \times (3 - 1.5)^2 + 2 \times (4.5 - 3)^2 = 9$
 - $Total = 1 + 9 = 10$
- Cohesion tăng mạnh (SSE giảm xuống 1), và Separation cũng tăng mạnh (SSB=9).
- **SSB + SSE = constant** → Điều này có nghĩa là trong bối cảnh này, việc **tối thiểu hóa SSE (tăng cohesion)** cũng đồng nghĩa với việc **tối đa hóa SSB (tăng separation)**.
- A **proximity graph-based approach** can also be used for cohesion and separation.
 - Cluster **cohesion** is **the sum of the weight of all links** within a cluster.
 - Cluster **separation** is **the sum of the weights between nodes in the cluster and nodes outside the cluster**.



cohesion



separation

5.6.1.2. Unsupervised Measures: Silhouette Coefficient

- Silhouette coefficient combines ideas of both **cohesion** and **separation**, **but for individual points**, as well as clusters and clusterings
- Với một điểm i:
 - **a**: Khoảng cách trung bình **từ i đến tất cả các điểm khác trong cùng cụm**. a đo lường **Cohesion**. a càng nhỏ, điểm i càng nằm "chặt" trong cụm của nó.
 - **b**: Khoảng cách trung bình **từ i đến tất cả các điểm trong cụm khác gần nhất**. b đo lường **Separation**. b càng lớn, cụm của i càng được phân tách tốt khỏi cụm lân cận.

- Công thức:
$$s = \frac{b - a}{\max(a, b)}, s \in [0, 1]$$

- Nếu **một điểm** được phân cụm tốt, b sẽ lớn hơn a rất nhiều ($b \gg a$), do đó **s sẽ tiến tới 1**.
- Nếu **một điểm** nằm ở biên giới giữa hai cụm, b sẽ xấp xỉ a ($b \approx a$), do đó **s sẽ tiến tới 0**.
- Nếu **một điểm** bị **gán sai cụm**, nó sẽ gần với cụm khác hơn là cụm của chính nó, tức là $a > b$, do đó **s sẽ là số âm**.
- Có thể tính hệ số đáng điều trung bình cho 1 cụm hoặc cho cả một phân cụm. Giá trị Silhouette trung bình cho toàn bộ dữ liệu càng gần 1 thì kết quả phân cụm càng tốt.

5.6.1.3. External index: Entropy, Purity

- Xây dựng bảng thống kê chéo **số phần tử của từng cụm thuộc từng lớp** đã có sẵn (k cụm, L lớp)
- Với mỗi cụm j, **tính xác suất để một phần tử thuộc cụm j (cluster j) thuộc về lớp i (class i):**

$$p_{ij} = \frac{|m_{ij}|}{|m_j|}$$

- $|m_{ij}|$: Số mẫu của class i thuộc cluster j
- $|m_j|$: Số mẫu của cluster j

Purity (Độ tinh khiết)

- Với một cụm bất kỳ, mức độ 'thuần nhất' của nó về mặt các lớp thực tế là bao nhiêu?
- Một cụm được coi là "hoàn toàn tinh khiết" nếu **tất cả các điểm trong đó đều thuộc về cùng một lớp thực tế.**

- **Công dụng**

- **Đo lường mức độ chính xác của từng cụm:** Cho biết mỗi cụm có tập trung tốt vào một lớp thực tế duy nhất hay không.
- **Dễ hiểu và dễ diễn giải:** Purity có giá trị từ 0 đến 1. Purity càng gần 1, kết quả phân cụm càng tốt. Purity = 1 là kết quả hoàn hảo.
- **So sánh nhanh các kết quả phân cụm:** Bạn có thể chạy nhiều thuật toán và chọn thuật toán có Purity tổng thể cao nhất.

- **Cách tính**

- **Bước 1: Tính Purity cho từng cụm (Purity_j)**

- Lấy một cụm j.
- Đếm xem có bao nhiêu điểm trong cụm j thuộc về mỗi lớp thực tế i (giá trị này là p_{ij}).
- Tìm ra lớp thực tế nào có nhiều điểm nhất trong cụm j (tức là tìm max(p_{ij}) qua tất cả các lớp i).
- **Purity (Độ thuần nhất) của cluster j:**

$$purity_j = \max_i p_{ij}$$

- **Purity của cụm j** = (số điểm của lớp chiếm đa số) / (tổng số điểm trong cụm j).

- **Bước 2: Tính Purity tổng thể (Overall Purity)**

- Purity tổng thể là **trung bình có trọng số** của Purity của tất cả các cụm. Trọng số chính là kích thước của mỗi cụm.

- **Tổng purity:**

$$purity = \sum_{j=1}^k \frac{|m_j|}{|m|} purity_j$$

- K: Số cụm (cluster)
- $|m_j|$: Số mẫu của cluster j
- $|m|$: Tổng số mẫu

- **Điểm yếu của Purity:** Purity có xu hướng "thường" cho việc tạo ra nhiều cụm nhỏ. Trong trường hợp cực đoan, nếu mỗi điểm là một cụm riêng, Purity sẽ bằng 1 một cách hoàn hảo, nhưng kết quả đó lại vô nghĩa.

Entropy

- Nó đo lường mức độ "lẫn lộn" hoặc "bất định" của các lớp thực tế bên trong một cụm.
- Với một cụm bất kỳ, mức độ 'hỗn loạn' về nhãn của các điểm bên trong nó là bao nhiêu?
- **Entropy = 0 (thấp nhất):** Cụm hoàn toàn "tinh khiết", không có sự lẫn lộn. Tất cả các điểm trong cụm đều thuộc về một lớp duy nhất. Đây là kết quả tốt nhất.
- **Entropy cao:** Cụm rất "hỗn loạn", các lớp được trộn lẫn đều với nhau. Đây là kết quả tồi.
- **Công dụng**
 - **Đo lường sự không chắc chắn:** Cung cấp một thước đo chặt chẽ về mặt toán học cho sự "lẫn lộn" của các nhãn trong một cụm.
 - **Nhạy cảm hơn Purity:** Entropy có thể phân biệt tốt hơn giữa các mức độ "hỗn loạn" khác nhau.
 - **Được sử dụng rộng rãi trong học máy:** Là một khái niệm nền tảng trong cây quyết định, lý thuyết thông tin, v.v.
- **Cách tính**
 - **Bước 1: Tính Entropy cho từng cụm (Entropy_j)**
 - Lấy một cụm j.
 - Với mỗi lớp thực tế i, tính xác suất p_{ij} để một điểm được chọn ngẫu nhiên từ cụm j sẽ thuộc về lớp i.
 - p_{ij} = (số điểm của lớp i trong cụm j) / (tổng số điểm trong cụm j).
 - **Entropy của cluster j:**

$$e_j = - \sum_{i=1}^L p_{ij} \log_2(p_{ij})$$

- Phép tính tổng được thực hiện trên tất cả các lớp i. Nếu p_{ij} = 0, thì p_{ij} * log₂(p_{ij}) được coi là 0.
- **Bước 2: Tính Entropy tổng thể (Overall Entropy)**
 - Tương tự như Purity, Entropy tổng thể là **trung bình có trọng số** của Entropy của tất cả các cụm.
 - **Tổng entropy:**

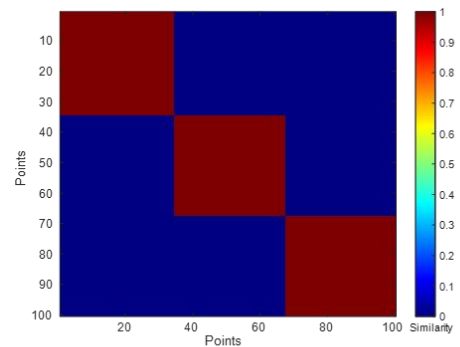
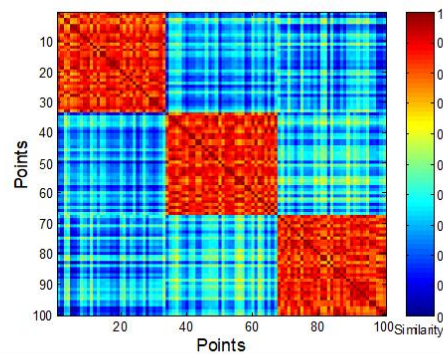
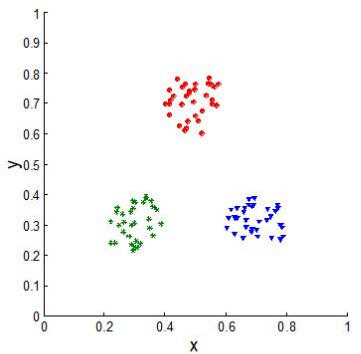
$$e = \sum_{j=1}^k \frac{|m_j|}{|m|} e_j$$

- L: Số lớp (class)
- K: Số cụm (cluster)
- |m_j|: Số mẫu của cluster j
- |m|: Tổng số mẫu

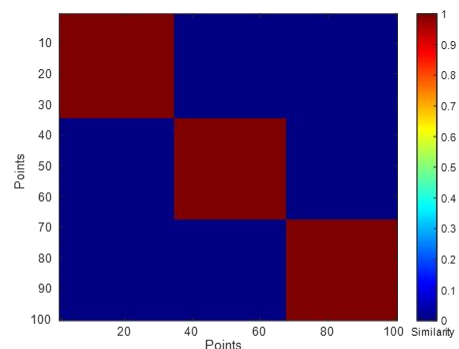
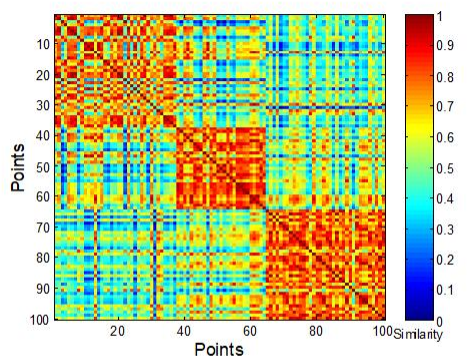
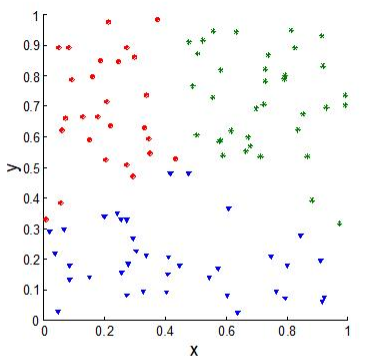
Tóm lại: Cả Purity và Entropy đều là những công cụ tuyệt vời để đánh giá hiệu suất của thuật toán phân cụm khi bạn may mắn có được dữ liệu đã được gán nhãn. Chúng cung cấp một cách định lượng để trả lời câu hỏi "**Thuật toán của tôi có tìm ra đúng các nhóm có ý nghĩa trong thực tế không?**".

5.6.2. Measuring Cluster Validity Via Correlation (Đo lường bằng Tương quan)

- **Ý tưởng:** Nếu một thuật toán phân cụm hoạt động tốt, thì các cặp điểm được xếp vào cùng một cụm **cũng phải là những cặp điểm có khoảng cách gần nhau trong dữ liệu gốc.**
- **Cách làm:**
 1. **Ideal Similarity Matrix (Ma trận Tương đồng Lý tưởng):** Tạo ra từ **kết quả phân cụm.** One row and one column for each data point. Ô (i, j) có giá trị 1 nếu điểm i và j cùng cụm, và 0 nếu khác cụm.
 2. **Proximity Matrix (Ma trận Khoảng cách):** **Tính từ dữ liệu gốc.** Ô (i, j) chứa khoảng cách giữa điểm i và j.
 3. **Tính tương quan:** Tính hệ số tương quan giữa hai ma trận này.
Since the matrices are symmetric (đối xứng), only the correlation between $n(n-1) / 2$ entries needs to be calculated.
- **Kết quả:** Nếu tương quan có **độ lớn cao (gần 1 hoặc -1)**, điều đó có nghĩa là cấu trúc mà thuật toán tìm thấy **khớp tốt** với cấu trúc khoảng cách tự nhiên của dữ liệu. (Nó sẽ là số âm nếu ma trận khoảng cách dùng "distance" và ma trận lý tưởng dùng "similarity", vì khoảng cách nhỏ tương ứng với tương đồng lớn).
- **Dữ liệu có cấu trúc tốt cho tương quan cao. Dữ liệu ngẫu nhiên cho tương quan thấp hơn nhiều.**



Corr = 0.9235



K-means

Corr = 0.5810

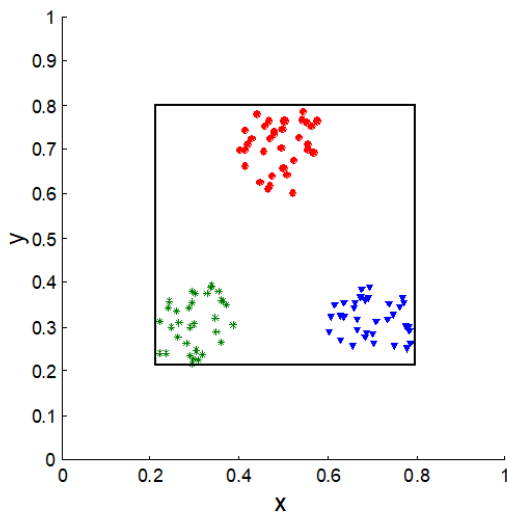
- **Not a good measure** for some **density or contiguity based clusters** (dựa trên mật độ hoặc liên kề).

- **Judging a Clustering Visually (Đánh giá Trực quan)**

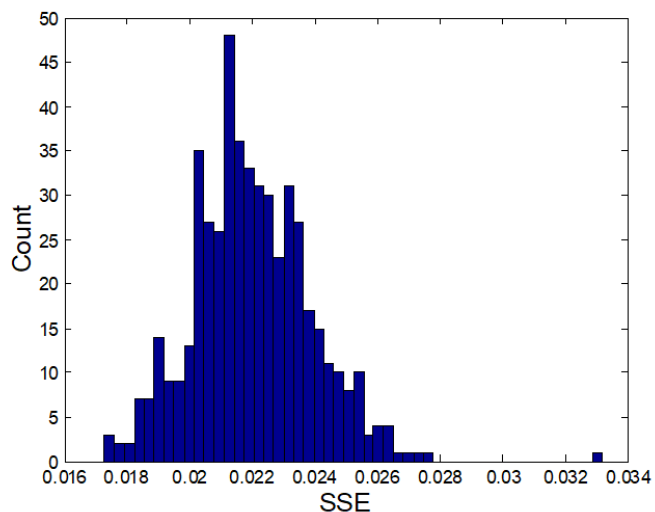
- Lấy ma trận khoảng cách (hoặc tương đồng) và sắp xếp lại các hàng và cột sao cho tất cả các điểm thuộc cùng một cụm nằm cạnh nhau.
- Nếu phân cụm tốt, bạn sẽ thấy các **khối vuông rõ rệt dọc theo đường chéo chính**.
- Bên trong mỗi khối vuông (intra-cluster), màu sắc phải đồng nhất và biểu thị sự tương đồng cao (ví dụ: màu nóng như đỏ/vàng).
- Bên ngoài các khối vuông (inter-cluster), màu sắc phải biểu thị sự tương đồng thấp (ví dụ: màu lạnh như xanh dương).

5.6.3. Statistical Framework (Khung đánh giá Thống kê)

- Trả lời câu hỏi: "Giá trị SSE của tôi là 0.005. Con số này có thực sự tốt không, hay chỉ là do may mắn?".
- **Ý tưởng:** So sánh kết quả của bạn trên dữ liệu **thật** với kết quả trên dữ liệu **ngẫu nhiên** (giả thuyết vô hiệu - null hypothesis).
- **Quy trình:**
 - **Tính độ đo trên dữ liệu thật của bạn** (ví dụ, $SSE = 0.005$).
 - Tạo ra hàng trăm bộ dữ liệu ngẫu nhiên có cùng kích thước (ví dụ 500) và phân bố như dữ liệu của bạn.
 - **Chạy cùng thuật toán phân cụm trên 500 bộ dữ liệu ngẫu nhiên đó** và tính SSE cho mỗi bộ.
 - Vẽ biểu đồ phân phối (histogram) của 500 giá trị SSE này. Biểu đồ này cho bạn thấy phạm vi giá trị SSE "bình thường" đối với dữ liệu ngẫu nhiên.



$SSE = 0.005$



Histogram shows SSE of three clusters in 500 sets of random data points of size 100 distributed over the range 0.2 – 0.8 for x and y values

- **So sánh:** Đặt giá trị SSE thật của bạn (0.005) lên biểu đồ này. Như hình trên, giá trị 0.005 nằm rất xa về bên trái so với toàn bộ phân phối (từ 0.016 đến 0.034).

- **Kết luận:** Xác suất để có được một giá trị SSE thấp như 0.005 một cách ngẫu nhiên là cực kỳ thấp. Do đó, chúng ta có thể kết luận rằng cấu trúc cụm mà chúng ta tìm thấy là **có ý nghĩa thống kê**, không phải là một sự trùng hợp ngẫu nhiên (tức là mô hình đã tìm được các nhóm thực sự trong dữ liệu chứ không phải chỉ là kết quả ngẫu nhiên).

5.6.4. Final Comment on Cluster Validity

- Trích dẫn từ Jain và Dubes, những người tiên phong trong lĩnh vực này, nói rằng **đánh giá cụm là phần khó nhất và gây nản lòng nhất**.
- "Without a strong effort in this direction, cluster analysis will remain a black art..." (Nếu không có nỗ lực mạnh mẽ theo hướng này, phân tích cụm sẽ vẫn là một nghệ thuật hắc ám...).
- **Thông điệp:** Việc chạy một thuật toán phân cụm rất dễ dàng. **Nhưng việc chứng minh rằng kết quả đó có ý nghĩa, đáng tin cậy và không phải là ngẫu nhiên đòi hỏi sự nghiêm túc, cẩn trọng và các phương pháp đánh giá vững chắc.** Đây là sự khác biệt giữa việc chỉ "sử dụng công cụ" và việc "làm khoa học dữ liệu" thực sự.

Tham khảo trực quan, chi tiết hơn qua các hình ảnh, các ví dụ:

<https://drive.google.com/file/d/1kvS-ElsfVA34cLpuP1QR3fjdNoH4gjGj/view>